

2.1. Data Manipulation

In order to get anything done, we need some way to store and manipulate data. Generally, there are two important things we need to do with data: (i) acquire them; and (ii) process them once they are inside the computer. There is no point in acquiring data without some way to store it, so let us get our hands dirty first by playing with synthetic data. To start, we introduce the n -dimensional array, which is also called the tensor.

If you have worked with a Linear Algebra library, then you will find this section familiar. No matter which framework you use, its tensor class (ndarray in MXNet) is similar to a Linear Algebra library with a few killer features. First, GPU is well-supported to accelerate the computation whereas Linear Algebra libraries only supports CPU computation. Second, the tensor class supports automatic differentiation. These properties make the tensor class suitable for deep learning. Throughout the book, when we say tensors, we are referring to instances of the tensor class unless otherwise stated.

2.1.1. Getting Started

In this section, we aim to get you up and running, equipping you with the basic math and numerical computing tools that you will build on as you progress through the book. Do not worry if you struggle to grok some of the mathematical concepts or library functions. The following sections will revisit this material in the context of practical examples and it will sink in. On the other hand, if you already have some background and want to go deeper into the mathematical content, just skip this section. To start, we import the modules from MXNet.

```
In [1]: # Todo programa que usted desarrolle debe cargar las siguientes librerías:
use strict;
use warnings;
use Data::Dump qw(dump);
use AI::MXNet qw(mx nd);
use sml;
```

```
In [2]: # Para plotear las gráficas, debe cargar las siguientes librerías:
use Chart::Plotly;
use Chart::Plotly::Plot;
use Chart::Plotly::Trace::Scatter;
IPerl->load_plugin('Chart::Plotly'); # (Jupyter) / Comentar si ejecuta desde IDE.
# use Chart::Plotly qw(show_plot); # Cuando si ejecuta desde IDE:
# show_plot($plot); # ejemplo: https://metacpan.org/pod/Chart::Plotly
```

```
In [3]: use Test::More;
$| = 1; # FORZAR AUTOFLUSH INMEDIATO

# Redirigir el output de Test::More directamente al STDOUT real de Jupyter
my $tb = Test::More->builder;
$tb->output(\*STDOUT);
$tb->failure_output(\*STDERR);
```

```
Out[3]: GLOB(0x55b32061f2a0)
```

A tensor represents a (possibly multi-dimensional) array of numerical values. With one axis, a tensor is called a vector. With two axes, a tensor is called a matrix. With $k > 2$ axes, we drop the specialized names and just refer to the object as a k th order tensor.

MXNet provides a variety of functions for creating new tensors prepopulated with values. For example, by invoking `arange(n)`, we can create a vector of evenly spaced values, starting at 0 (included) and ending at `n` (not included). By default, the interval size is 1. Unless otherwise specified, new tensors are stored in main memory and designated for CPU-based computation.

```
In [4]: nd->find_functions('*norm*')
```

```
0 _backward_broadcast_normal
1 _backward_image_normalize
2 _backward_norm
3 _backward_npi_norm
4 _backward_pdf_normal
5 _contrib_quantized_batch_norm
6 _image_normalize
7 _npi_norm
8 _npi_normal
9 _npi_normal_n
10 _npx__image_normalize
11 _npx_batch_norm
12 _npx_layer_norm
13 _random_normal
14 _random_normal_like
15 _random_pdf_normal
16 _sample_normal
17 _sparse_norm
18 norm
19 normal
20 random_normal
21 random_pdf_normal
22 sample_normal
```

```
Out[4]: HASH(0x55b32717a798)HASH(0x55b3271abee8)HASH(0x55b3271e4028)HASH(0x55b327213d50)HASH
(0x55b32722b0d8)HASH(0x55b3272db350)HASH(0x55b327317748)HASH(0x55b327450908)HASH(0x55
b327453610)HASH(0x55b327454060)HASH(0x55b3274c1370)HASH(0x55b3274d82d0)HASH(0x55b3274
f6738)HASH(0x55b327532ed0)HASH(0x55b327534a88)HASH(0x55b32753e998)HASH(0x55b32755d30
0)HASH(0x55b3275c00a8)HASH(0x55b3276b5248)HASH(0x55b3276b5c50)HASH(0x55b3276d2690)HAS
H(0x55b3276daa50)HASH(0x55b3277049a0)
```

```
In [5]: nd->help('broadcast_to')
```

Function: `_npi_broadcast_to`

Parameters

`array` : NDAarray-or-Symbol

The input

`shape` : Shape(tuple), optional, default=[]

The shape of the desired array. We can set the dim to zero if it's same as the original. E.g ``A = broadcast_to(B, shape=(10, 0, 0))`` has the same meaning as ``A = broadcast_axis(B, axis=0, size=10)``.

`out` : NDAarray, optional

The output NDAarray to hold the result.

Returns

`out` : NDAarray or list of NDAarray

The output of this function.

=====

Function: `broadcast_to`

Broadcasts the input array to a new shape.

Broadcasting is a mechanism that allows NDAarrays to perform arithmetic operations with arrays of different shapes efficiently without creating multiple copies of arrays.

Also see, ``Broadcasting <https://docs.scipy.org/doc/numpy/user/basics.broadcasting.html>`` for more explanation.

Broadcasting is allowed on axes with size 1, such as from ``(2,1,3,1)`` to ``(2,8,3,9)``. Elements will be duplicated on the broadcasted axes.

For example::

```
broadcast_to([[1,2,3]], shape=(2,3)) = [[ 1.,  2.,  3.],
                                           [ 1.,  2.,  3.]]
```

The dimension which you do not want to change can also be kept as ``0`` which means copy the original value.

So with ``shape=(2,0)``, we will obtain the same result as in the above example.

Defined in `../src/operator/tensor/broadcast_reduce_op_value.cc:L116`

Parameters

`data` : NDAarray-or-Symbol

The input

`shape` : Shape(tuple), optional, default=[]

The shape of the desired array. We can set the dim to zero if it's same as the original. E.g ``A = broadcast_to(B, shape=(10, 0, 0))`` has the same meaning as ``A = broadcast_axis(B, axis=0, size=10)``.

`out` : NDAarray, optional

The output NDAarray to hold the result.

Returns

out : NDArray or list of NDArray
The output of this function.

=====

```
In [6]: my $update = nd->array(
        [ 1.0, -2.0, 3.0, -4.0, 5.0 ],
        dtype => 'float64'
      );

my $grad = nd->array(
        [ -1.0, -3.0, 2.0, 4.0, 5.0 ],
        dtype => 'float64'
      );

print "update = ", $update->asf, "\n";
print "grad   = ", $grad->asf, "\n";

my $prod = $update * $grad;
print "producto = ", $prod->asf, "\n";

my $inc = ($prod < 0.0);
print "inc = ", $inc->asf, "\n";

my $dec = nd->logical_not($inc);
print "dec = ", $dec->asf, "\n";

# update = [1 -2 3 -4 5]
# grad   = [-1 -3 2 4 5]
# producto = [-1 6 6 -16 25]
# inc = [1 0 0 1 0]
# dec = [0 1 1 0 1]
```

```
update = [1 -2 3 -4 5]
grad   = [-1 -3 2 4 5]
producto = [-1 6 6 -16 25]
inc = [1 0 0 1 0]
dec = [0 1 1 0 1]
```

Out[6]: 1

```
In [7]: my $x = nd->arange(start => 0,
                          stop  => 12,
                          step  => 1,
                          ctx   => mx->cpu(0));
printf "%s\n", $x->asstr;
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

Out[7]: 1

```
In [8]: # Tipo booleano
my $result = nd->isfinite($x);
nd->print($result);
```

```
[1 1 1 1 1 1 1 1 1 1 1 1]
```

```
<AI::MXNet::NDArray 12 @cpu(0)> AI::MXNet::NDArray bool
```

Out[8]: 1

```
In [9]: # Tipo booleano
print "True" if nd->all(nd->isfinite($x))->asscalar;
```

```
True
```

Out[9]: 1

```
In [10]: $x->power(2)->print;

[ 0  1  4  9 16 25 36 49 64 81 100 121]
<AI::MXNet::NDArray 12 @cpu(0)> AI::MXNet::NDArray float32
```

Out[10]: 1

```
In [11]: my $y = $x->as_in_context(mx->cpu(1));
printf "%s\n", $y;
print $y->asstr;
```

```
<AI::MXNet::NDArray 12 @cpu(1)>
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

Out[11]: 1

```
In [12]: my $z = $x + $y;
print $z->asstr;
```

```
[ 0  2  4  6  8 10 12 14 16 18 20 22]
```

Out[12]: 1

```
In [13]: $y = nd->arange(12)->reshape([3, 4]);
print $y, $y->asstr;
$x = $y->transpose();
print $x, $x->asstr;
```

```
<AI::MXNet::NDArray 3x4 @cpu(0)>[[ 0  1  2  3]
[ 4  5  6  7]
[ 8  9 10 11]]<AI::MXNet::NDArray 4x3 @cpu(0)>[[ 0  4  8]
[ 1  5  9]
[ 2  6 10]
[ 3  7 11]]
```

Out[13]: 1

```
In [14]: $y = nd->arange(16)->reshape([4, 4]);
nd->tril($y, k=>-1)->aspdf
```

```
Out[14]: [
[ 0  0  0  0]
[ 4  0  0  0]
[ 8  9  0  0]
[12 13 14  0]
]
```

```
In [15]: nd->tril($y, k=>-1)->nonzero()->aspdf
```

```
Out[15]: [
[1 0]
[2 0]
[2 1]
[3 0]
[3 1]
[3 2]
]
```

```
In [16]: my $indices = nd->tril_indices(4, -1);
```

```
nd->print($indices);
# [[1 2 2 3 3 3]
#  [0 0 1 0 1 2]]
```

```
[[1 2 2 3 3 3]
[0 0 1 0 1 2]]
<AI::MXNet::NDArray 2x6 @cpu(0)> AI::MXNet::NDArray int64
```

Out[16]: 1

```
In [17]: nd->gather_nd($y, $indices)->print;
```

```
[ 4  8  9 12 13 14]
<AI::MXNet::NDArray 6 @cpu(0)> AI::MXNet::NDArray float32
```

Out[17]: 1

```
In [18]: $indices = nd->triu_indices(4, 1);
nd->print($indices);
# i = [0 0 0 1 1 2]
# j = [1 2 3 2 3 3]
```

```
[[0 0 0 1 1 2]
 [1 2 3 2 3 3]]
<AI::MXNet::NDArray 2x6 @cpu(0)> AI::MXNet::NDArray int64
```

Out[18]: 1

```
In [19]: my $y = nd->arange(12)->reshape([3, 4]);
my $indices = nd->triu_indices(4, 1);

my $condensed = nd->gather_nd($y, $indices);
nd->print($y, $condensed);

# Result:
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]
#  [12 13 14 15]]
# <AI::MXNet::NDArray 4x4 @cpu(0)> AI::MXNet::NDArray float32
# [ 1  2  3  6  7 11]
# <AI::MXNet::NDArray 6 @cpu(0)> AI::MXNet::NDArray float32
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray float32
[ 1  2  3  6  7 11]
<AI::MXNet::NDArray 6 @cpu(0)> AI::MXNet::NDArray float32
```

Out[19]: 1

```
In [20]: $x = nd->arange(12)->reshape([3,4]);
nd->printf("x:%s\n", $x);
$y = nd->arange(3)->reshape([3,1]);
nd->printf("y:%s\n", $y);
$z = $x + $y;
nd->printf("z:%s\n", $z);
```

```
x:[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray float32
```

```
y:[[0]
 [1]
 [2]]
<AI::MXNet::NDArray 3x1 @cpu(0)> AI::MXNet::NDArray float32
```

```
z:[[ 0  1  2  3]
 [ 5  6  7  8]
 [10 11 12 13]]
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray float32
```

Out[20]: 1

```
In [21]: $y = nd->array([2]);
nd->printf("y:%s\n", $y);
$z = $x + $y;
nd->printf("z:%s\n", $z);
```

y:[2]

<AI::MXNet::NDArray 1 @cpu(0)> AI::MXNet::NDArray float32

z:[[2 3 4 5]

[6 7 8 9]

[10 11 12 13]]

<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray float32

Out[21]: 1

```
In [22]: $x = nd->arange(12, step => 0.5);
print $x->asstr;
```

```
[ 0 0.5000 1 1.5000 2 2.5000 3 3.5000 4 4.5000
5 5.5000 6 6.5000 7 7.5000 8 8.5000 9 9.5000 10 10.5
000 11 11.5000]
```

Out[22]: 1

```
In [23]: print $y->dtype;
```

float32

Out[23]: 1

```
In [24]: $y = $y->astype('int32');
print $y->dtype;
```

int32

Out[24]: 1

```
In [25]: my $z = nd->array([16.9, 8.0, 10.5], dtype=>'int32');
printf "%s\n", $z->dtype;
printf "%s\n", $z->asstr;
```

int32

[16 8 10]

Out[25]: 1

```
In [26]: print $x;
```

<AI::MXNet::NDArray 24 @cpu(0)>

Out[26]: 1

```
In [27]: print ref($x);
```

AI::MXNet::NDArray

Out[27]: 1

```
In [28]: print dump $x;
```

```
bless({
  handle => bless(do{\(my $o = 94228041909632)}, "NDArrayHandle"),
  writable => 1,
}, "AI::MXNet::NDArray")
```

Out[28]: 1

```
In [29]: print $x->handle;
```

```
NDArrayHandle=SCALAR(0x55b32d000a20)
```

```
Out[29]: 1
```

We can access a tensor's shape (the length along each axis) by inspecting its shape property.

```
In [30]: $x->shape; # Devuelve una referencia a un arreglo Perl
```

```
Out[30]: ARRAY(0x55b32d01e100)
```

```
In [31]: print dump $x->shape;
```

```
[24]
```

```
Out[31]: 1
```

```
In [32]: print $x->shape->[0];
```

```
24
```

```
Out[32]: 1
```

```
In [33]: my ($rows, $cols) = @{$x->shape};  
print dump ($rows, $cols);
```

```
(24, undef)
```

```
Out[33]: 1
```

```
In [34]: print $x->len; # Sinónimo de: $x->shape->[0];
```

```
24
```

```
Out[34]: 1
```

```
In [35]: print $x->ndim;
```

```
1
```

```
Out[35]: 1
```

If we just want to know the total number of elements in a tensor, i.e., the product of all of the shape elements, we can inspect its size. Because we are dealing with a vector here, the single element of its shape is identical to its size.

```
In [36]: print $x->size;
```

```
24
```

```
Out[36]: 1
```

To change the shape of a tensor without altering either the number of elements or their values, we can invoke the reshape function. For example, we can transform our tensor, x, from a row vector with shape (12,) to a matrix with shape (3, 4). This new tensor contains the exact same values, but views them as a matrix organized as 3 rows and 4 columns. To reiterate, although the shape has changed, the elements have not. Note that the size is unaltered by reshaping.

```
In [37]: $x = nd->arange(start => 0, stop => 12)->reshape([3, 4]);  
print $x->asstr;
```

```
[[ 0  1  2  3]  
 [ 4  5  6  7]  
 [ 8  9 10 11]]
```

```
Out[37]: 1
```

```
In [38]: print $x->reshape([-1, 2])>asstr;
```

```
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [ 8  9]
 [10 11]]
```

Out[38]: 1

```
In [39]: print $x->transpose([0, 1])>asstr;
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Out[39]: 1

```
In [40]: $x = nd->arange(start => 0, stop => 24);
         $x = $x->reshape([2, 3, 4]);
         print $x->asstr;
```

```
[[[ 0  1  2  3]
   [ 4  5  6  7]
   [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
```

Out[40]: 1

```
In [41]: print $x->reshape([$x->len, -1])>asstr;
```

```
[[ 0  1  2  3  4  5  6  7  8  9 10 11]
 [12 13 14 15 16 17 18 19 20 21 22 23]]
```

Out[41]: 1

```
In [42]: $x = nd->arange(24)->reshape([2,3,4]);
         print $x->asstr;
```

```
[[[ 0  1  2  3]
   [ 4  5  6  7]
   [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
```

Out[42]: 1

```
In [43]: print $x->transpose([1,0,2])>asstr;
```

```
[[[ 0  1  2  3]
   [12 13 14 15]]

 [[ 4  5  6  7]
  [16 17 18 19]]

 [[ 8  9 10 11]
  [20 21 22 23]]]
```

Out[43]: 1

```
In [44]: print $x->transpose([1, 2, 0])>asstr;
```

```

[[[ 0 12]
  [ 1 13]
  [ 2 14]
  [ 3 15]]

 [[ 4 16]
  [ 5 17]
  [ 6 18]
  [ 7 19]]

 [[ 8 20]
  [ 9 21]
 [10 22]
 [11 23]]]

```

Out[44]: 1

```
In [45]: print $x->transpose()->asstr;
```

```

[[[ 0 12]
  [ 4 16]
  [ 8 20]]

 [[ 1 13]
  [ 5 17]
  [ 9 21]]

 [[ 2 14]
  [ 6 18]
 [10 22]]

 [[ 3 15]
  [ 7 19]
 [11 23]]]

```

Out[45]: 1

```
In [46]: $y = $x->transpose([1, 2, 0]);
print $y->asstr;
```

```

[[[ 0 12]
  [ 1 13]
  [ 2 14]
  [ 3 15]]

 [[ 4 16]
  [ 5 17]
  [ 6 18]
  [ 7 19]]

 [[ 8 20]
  [ 9 21]
 [10 22]
 [11 23]]]

```

Out[46]: 1

```
In [47]: print dump $y->shape;
```

```
[3, 4, 2]
```

Out[47]: 1

Reshaping by manually specifying every dimension is unnecessary. If our target shape is a matrix with shape (height, width), then after we know the width, the height is given implicitly. Why should we have to perform the division ourselves? In the example above, to get a matrix with 3 rows, we specified both that it should have 3 rows and 4 columns. Fortunately, tensors can automatically work out one dimension

given the rest. We invoke this capability by placing -1 for the dimension that we would like tensors to automatically infer. In our case, instead of calling `$X->reshape([3, 4])`, we could have equivalently called `$X->reshape([-1, 4])`, or `$X->reshape([3, -1])`.

Typically, we will want our matrices initialized either with zeros, ones, some other constants, or numbers randomly sampled from a specific distribution. We can create a tensor representing a tensor with all elements set to 0 and a shape of (2, 3, 4) as follows:

```
In [48]: my $var = nd->zeros([2, 3, 4]);
```

```
Out[48]: <AI::MXNet::NDArray 2x3x4 @cpu(0)>
```

```
In [49]: print $var->asstr;
```

```
[[[0 0 0 0]
  [0 0 0 0]
  [0 0 0 0]]

 [[0 0 0 0]
  [0 0 0 0]
  [0 0 0 0]]]
```

```
Out[49]: 1
```

Similarly, we can create tensors with each element set to 1 as follows:

```
In [50]: $var = nd->ones([2, 3, 4]);
```

```
Out[50]: <AI::MXNet::NDArray 2x3x4 @cpu(0)>
```

```
In [51]: print $var->asstr;
```

```
[[[1 1 1 1]
  [1 1 1 1]
  [1 1 1 1]]

 [[1 1 1 1]
  [1 1 1 1]
  [1 1 1 1]]]
```

```
Out[51]: 1
```

```
In [52]: $var = nd->full([2, 3, 4], 1e-9);
print $var->asstr;
```

```
[[[0.0000 0.0000 0.0000 0.0000]
  [0.0000 0.0000 0.0000 0.0000]
  [0.0000 0.0000 0.0000 0.0000]]

 [[0.0000 0.0000 0.0000 0.0000]
  [0.0000 0.0000 0.0000 0.0000]
  [0.0000 0.0000 0.0000 0.0000]]]
```

```
Out[52]: 1
```

Often, we want to randomly sample the values for each element in a tensor from some probability distribution. For example, when we construct arrays to serve as parameters in a neural network, we will typically initialize their values randomly. The following snippet creates a tensor with shape (3, 4). Each of its elements is randomly sampled from a standard Gaussian (normal) distribution with a mean of 0 and a standard deviation of 1.

```
In [53]: print nd->random_normal(10, 5, shape => [3, 4])->asstr;
```

```
[[21.0610 13.8700 15.2172 15.9196]
 [19.4586  3.8263  1.1449  7.7431]
 [12.8969  0.7196  0.1156  8.9599]]
```

Out[53]: 1

```
In [54]: print nd->random->normal(loc => 0, scale => 1, shape => [3, 4])->asstr;
```

```
[[ 0.2444 -0.0372 -0.4877 -0.0226]
 [ 0.5746  1.4661  0.6863  0.3550]
 [ 1.0732  0.1202 -0.9711 -0.7757]]
```

Out[54]: 1

We can also specify the exact values for each element in the desired tensor by supplying a Perl array reference containing the numerical values. Here, the outermost array corresponds to axis 0, and the inner list to axis 1.

```
In [55]: print nd->array([[2, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])->asstr;
```

```
[[2 1 4 3]
 [1 2 3 4]
 [4 3 2 1]]
```

Out[55]: 1

```
In [56]: $x = nd->array([[2, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]]);
print $x->asstr;
```

```
[[2 1 4 3]
 [1 2 3 4]
 [4 3 2 1]]
```

Out[56]: 1

```
In [57]: my @array = (1, 2, 3);
print nd->array(\@array)->asstr;
```

```
[1 2 3]
```

Out[57]: 1

```
In [58]: my $array2 = [4, 5, 6];
my $nd_arr = nd->array($array2);
```

Out[58]: <AI::MXNet::NDArray 3 @cpu(0)>

```
In [59]: print $nd_arr->asstr;
```

```
[4 5 6]
```

Out[59]: 1

```
In [60]: my $arr2 = [[2, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]];
```

Out[60]: ARRAY(0x55b32d047010)

```
In [61]: print nd->array($arr2)->asstr;
```

```
[[2 1 4 3]
 [1 2 3 4]
 [4 3 2 1]]
```

Out[61]: 1

```
In [62]: my $array2 = $x->asarray;
print dump $array2;
```

```
[[2, 1, 4, 3], [1 .. 4], [4, 3, 2, 1]]
```

Out[62]: 1

```
In [63]: my @lista2 = $x->tolist;  
         print dump @lista2;
```

```
([2, 1, 4, 3], [1 .. 4], [4, 3, 2, 1])
```

Out[63]: 1

Shared axis slice function

There is a slice function which works with parameters begin, end, [step]. These parameters can be a scalar or an array. In case of scalars, they simply mean the begin, end or step acting on the first axis. Otherwise as an array, the index of their components indicate the respective axis. In this sense, the axis positions is shared among the three parameters. Finally, notice that the last position located in 'end' is not included, just like Python's range.

```
In [64]: my $x = nd->arange(30)->reshape([5, 6]);  
         nd->print($x);
```

```
[[ 0  1  2  3  4  5]  
 [ 6  7  8  9 10 11]  
[12 13 14 15 16 17]  
[18 19 20 21 22 23]  
[24 25 26 27 28 29]]
```

```
<AI::MXNet::NDArray 5x6 @cpu(0)> AI::MXNet::NDArray float32
```

Out[64]: 1

```
In [65]: nd->print($x->slice(begin=>[0,], end=>[2,]));
```

```
[[ 0  1  2  3  4  5]  
 [ 6  7  8  9 10 11]]
```

```
<AI::MXNet::NDArray 2x6 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[65]: 1

```
In [66]: nd->print($x->slice(begin=>[0,], end=>[3,], step=>2));
```

```
[[ 0  2  4]  
[12 14 16]]
```

```
<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[66]: 1

```
In [67]: nd->print($x->slice(begin=>[0, 0], end=>[3, 6], step=>[2, 2]));
```

```
[[ 0  2  4]  
[12 14 16]]
```

```
<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[67]: 1

```
In [68]: my $y = $x->slice(begin=>[0, 0], end=>[3, 6], step=>[2, 2]);  
         nd->print($y);
```

```
[[ 0  2  4]  
[12 14 16]]
```

```
<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[68]: 1

```
In [69]: # Odd columns  
         nd->print($x->slice(begin=>[0, 0], end=>undef, step=>[1, 2]));
```

```

[[ 0  2  4]
 [ 6  8 10]
 [12 14 16]
 [18 20 22]
 [24 26 28]]
<AI::MXNet::NDArray 5x3 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[69]: 1

```

In [70]: # Even columns
nd->print($x->slice(begin=>[0, 1], end=>undef, step=>[1, 2]));

```

```

[[ 1  3  5]
 [ 7  9 11]
 [13 15 17]
 [19 21 23]
 [25 27 29]]
<AI::MXNet::NDArray 5x3 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[70]: 1

```

In [71]: # Shared axis slice function allows an assignment to change some tensor values.

```

```

In [72]: nd->print($x->slice(begin=>[0, 0], end=>[undef, 2]));

```

```

[[ 0  1]
 [ 6  7]
 [12 13]
 [18 19]
 [24 25]]
<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[72]: 1

```

In [73]: my $y = $x->slice(begin=>[0, 0], end=>[undef, 2]);
nd->print($y);
nd->print($y->sin());

```

```

[[ 0  1]
 [ 6  7]
 [12 13]
 [18 19]
 [24 25]]
<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[ 0.0000  0.8415]
 [-0.2794  0.6570]
 [-0.5366  0.4202]
 [-0.7510  0.1499]
 [-0.9056 -0.1324]]
<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray float32

```

Out[73]: 1

```

In [74]: $x->slice(begin=>[0, 0], end=>[undef, 2])>set($y->sin());
nd->print($x); # tensor is changed.

```

```

[[ 0.0000  0.8415      2      3      4      5]
 [-0.2794  0.6570      8      9     10     11]
 [-0.5366  0.4202     14     15     16     17]
 [-0.7510  0.1499     20     21     22     23]
 [-0.9056 -0.1324     26     27     28     29]]
<AI::MXNet::NDArray 5x6 @cpu(0)> AI::MXNet::NDArray float32

```

Out[74]: 1

Full axis slice function ':'

There is a FULL axis slice function does not contain the words begin/end/step as the shared axis. Instead, it represents a whole axis with an 'X' or ':' like in Python. If the tensor is unidimensional, the begin and end are just scalars. For the multidimensional tensors, we place the indexes into brackets.

```
In [75]: $x = nd->arange(30)->reshape([5, 6]);
nd->print($x->slice(begin=>[0, 0], end=>[3, 6]));

[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]]
<AI::MXNet::NDArray 3x6 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[75]: 1
```

```
In [76]: # Each parameter handles full axis: [begin, and] together for the full axis.
nd->print($x->slice('X', [0, 2]));

[[ 0  1]
 [ 6  7]
 [12 13]
 [18 19]
 [24 25]]
<AI::MXNet::NDArray 5x2 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[76]: 1
```

```
In [77]: # Allows assignment by using slice()->set()
$x->slice('X', [0, 2])->set($y->sin());
nd->print($x);

[[ 0.0000  0.7456      2      3      4      5]
 [-0.2758  0.6107      8      9     10     11]
 [-0.5112  0.4079     14     15     16     17]
 [-0.6824  0.1493     20     21     22     23]
 [-0.7868 -0.1320     26     27     28     29]]
<AI::MXNet::NDArray 5x6 @cpu(0)> AI::MXNet::NDArray float32

Out[77]: 1
```

```
In [78]: $x = nd->arange(12)->reshape([3, 4]);
nd->print($x);

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray float32

Out[78]: 1
```

```
In [79]: $x->slice([0, 2], [2, 4]) .= nd->full([2, 2], 10);
nd->print($x);

[[ 0  1 10 10]
 [ 4  5 10 10]
 [ 8  9 10 11]]
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray float32

Out[79]: 1
```

slice_axis function

When we need a slice of a **particular axis**, we can use the slice_axis function. It slices along a given axis. It returns an array slice along a given axis starting from the begin index to the end index which is not included. The result is a AI::MXNet::NDArray tensor.

```
In [80]: nd->print(nd->slice_axis($x, axis=>0, begin=>1, end=>3));
```

```
[[ 4  5 10 10]
 [ 8  9 10 11]]
<AI::MXNet::NDArray 2x4 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[80]: 1
```

```
In [81]: nd->print(nd->slice_axis($x, axis=>1, begin=>1, end=>4));
```

```
[[ 1 10 10]
 [ 5 10 10]
 [ 9 10 11]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[81]: 1
```

Compatibility with Python NDArray slices

Henceforth, we show the syntax used in both Perl MXNet and Python's Numpy. The expected results should be the same.

```
In [82]: #
# 1. Backward Compatibility Tests
#
```

```
my $A = nd->arange(10);
nd->print($A->slice(0));      # A[0]          => [0]
nd->print($A->slice(5));      # A[5]          => [5]
nd->print($A->slice('X'));     # A[:]          => [0 1 2 3 4 5 6 7 8 9]
```

```
[0]
<AI::MXNet::NDArray 1 @cpu(0)> AI::MXNet::NDArray::Slice float32
[5]
<AI::MXNet::NDArray 1 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 1 2 3 4 5 6 7 8 9]
<AI::MXNet::NDArray 10 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

```
Out[82]: 1
```

```
In [83]: #
# 2.1 Colon Tests (1D) with indices
#
```

```
nd->print($A->slice(':'));     # A[:]          => [0 1 2 3 4 5 6 7 8 9]
nd->print($A->slice([1,5]));    # A[1:5]        => [1 2 3 4]
nd->print($A->slice([0,3]));    # A[0:3]        => [0 1 2]
nd->print($A->slice([5,10]));   # A[5:10]       => [5 6 7 8 9]
nd->print($A->slice([-3,10]));   # A[-3:10]      => [7 8 9]
nd->print($A->slice([0,-1]));   # A[0:-1]       => [0 1 2 3 4 5 6 7 8]
```

```
[0 1 2 3 4 5 6 7 8 9]
<AI::MXNet::NDArray 10 @cpu(0)> AI::MXNet::NDArray::Slice float32
[1 2 3 4]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 1 2]
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray::Slice float32
[5 6 7 8 9]
<AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray::Slice float32
[7 8 9]
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 1 2 3 4 5 6 7 8]
<AI::MXNet::NDArray 9 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

```
Out[83]: 1
```

```
In [84]: #
# 2.2 Colon Tests (1D) with undef
```

```
#
$A = nd->arange(10);
nd->print($A->slice([3, 5]));          # A[3:5] => [3 4]
nd->print($A->slice([undef, undef]));    # A[:]   => [0 1 2 3 4 5 6 7 8 9]
nd->print($A->slice([3, undef]));        # A[3:]  => [3 4 5 6 7 8 9]
nd->print($A->slice([undef, 5]));        # A[:5]  => [0 1 2 3 4]
```

```
[3 4]
<AI::MXNet::NDArray 2 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 1 2 3 4 5 6 7 8 9]
<AI::MXNet::NDArray 10 @cpu(0)> AI::MXNet::NDArray::Slice float32
[3 4 5 6 7 8 9]
<AI::MXNet::NDArray 7 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 1 2 3 4]
<AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[84]: 1

```
In [85]: #
# 3.1 Step Tests (1D) with indices
#
nd->print($A->slice([1,8,2]));          # A[1:8:2] => [1 3 5 7]
nd->print($A->slice([0,10,3]));         # A[0:10:3] => [0 3 6 9]
nd->print($A->slice([2,9,2]));          # A[2:9:2] => [2 4 6 8]
```

```
[1 3 5 7]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 3 6 9]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float32
[2 4 6 8]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[85]: 1

```
In [86]: #
# 3.2 Step Tests (1D) with undef
#
nd->print($A->slice([3, 5, 2]));          # A[3:5:2] => [3]
nd->print($A->slice([undef, undef, 2]));  # A[:,2]   => [0 2 4 6 8]
nd->print($A->slice([3, undef, 2]));      # A[3::2]  => [3 5 7 9]
nd->print($A->slice([undef, 5, 2]));      # A[:5:2]  => [0 2 4]
```

```
[3]
<AI::MXNet::NDArray 1 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 2 4 6 8]
<AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray::Slice float32
[3 5 7 9]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 2 4]
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[86]: 1

```
In [87]: #
# 4. begin/end Compatibility Tests (1D)
#
nd->print($A->slice(begin=>undef, end=>undef)); # A[:]   => [0 1 2 3 4 5 6 7 8 9]
nd->print($A->slice(begin=>3, end=>undef));      # A[3:]  => [3 4 5 6 7 8 9]
nd->print($A->slice(begin=>undef, end=>5));      # A[:5]  => [0 1 2 3 4]
nd->print($A->slice(begin=>3, end=>5));          # A[3:5]  => [3 4]
nd->print($A->slice(begin=>1, end=>8, step=>2)); # A[1:8:2] => [1 3 5 7]
```

```
[0 1 2 3 4 5 6 7 8 9]
<AI::MXNet::NDArray 10 @cpu(0)> AI::MXNet::NDArray::Slice float32
[3 4 5 6 7 8 9]
<AI::MXNet::NDArray 7 @cpu(0)> AI::MXNet::NDArray::Slice float32
[0 1 2 3 4]
<AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray::Slice float32
[3 4]
<AI::MXNet::NDArray 2 @cpu(0)> AI::MXNet::NDArray::Slice float32
[1 3 5 7]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[87]: 1

```
In [88]: #
# 5. Basic 2D Tests with indices
#
$A = nd->arange(20)->reshape([4,5]);
nd->print($A->slice([0,2],':')); # A[0:2,:] First two rows
nd->print($A->slice([2,4],':')); # A[2:4,:] Last two rows
nd->print($A->slice(2,':')); # A[2,:] [10 11 12 13 14] # Single row
nd->print($A->slice(':',2)); # A[:,2] [ 2  7 12 17] # Single column
```

```
[[0 1 2 3 4]
 [5 6 7 8 9]]
<AI::MXNet::NDArray 2x5 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[10 11 12 13 14]
 [15 16 17 18 19]]
<AI::MXNet::NDArray 2x5 @cpu(0)> AI::MXNet::NDArray::Slice float32
[10 11 12 13 14]
<AI::MXNet::NDArray 5 @cpu(0)> AI::MXNet::NDArray::Slice float32
[ 2  7 12 17]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[88]: 1

```
In [89]: #
# 5. Basic 2D Tests with undef
#
nd->print($A->slice(':', [0, undef, 2])); # A[:,0::2]
nd->print($A->slice([undef, 4], ':')); # A[:,4,:]
nd->print($A->slice([undef, undef, 2], ':')); # A[:,2,:]
nd->print($A->slice(':', [undef, undef, 3])); # A[:,::3]
nd->print($A->slice([1, undef], [0, undef, 2])); # A[1:, 0::2]
```

```
[[ 0  2  4]
 [ 5  7  9]
 [10 12 14]
 [15 17 19]]
<AI::MXNet::NDArray 4x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
<AI::MXNet::NDArray 4x5 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[ 0  1  2  3  4]
 [10 11 12 13 14]]
<AI::MXNet::NDArray 2x5 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[ 0  3]
 [ 5  8]
 [10 13]
 [15 18]]
<AI::MXNet::NDArray 4x2 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[ 5  7  9]
 [10 12 14]
 [15 17 19]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[89]: 1

```
In [90]: nd->print($A->slice(':', ':'));
# A[:, :]
# [[ 0  1  2  3  4]
#   [ 5  6  7  8  9]
#  [10 11 12 13 14]
#  [15 16 17 18 19]]

nd->print($A->slice([1,4], ':'));
# A[1:4, :]
# [[ 5  6  7  8  9]
#   [10 11 12 13 14]
#   [15 16 17 18 19]]

nd->print($A->slice(':', [1,4]));
# A[:, 1:4]
# [[ 1  2  3]
#   [ 6  7  8]
#  [11 12 13]
#  [16 17 18]]

nd->print($A->slice([1,3], [1,4]));
# A[1:3, 1:4]
# [[ 6  7  8]
#   [11 12 13]]
```

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
```

<AI::MXNet::NDArray 4x5 @cpu(0)> AI::MXNet::NDArray::Slice float32

```
[[ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
```

<AI::MXNet::NDArray 3x5 @cpu(0)> AI::MXNet::NDArray::Slice float32

```
[[ 1  2  3]
 [ 6  7  8]
 [11 12 13]
 [16 17 18]]
```

<AI::MXNet::NDArray 4x3 @cpu(0)> AI::MXNet::NDArray::Slice float32

```
[[ 6  7  8]
 [11 12 13]]
```

<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32

Out[90]: 1

```
In [91]: #
# 6. Step Tests (2D)
#
#=pod
nd->print($A->slice([0,4,2], ':'));
# A[0:4:2, :]
# [[ 0  1  2  3  4]
#   [10 11 12 13 14]]

nd->print($A->slice(':', [0,5,2]));
# A[:, 0:5:2]
# [[ 0  2  4]
#   [ 5  7  9]
#  [10 12 14]
#  [15 17 19]]

nd->print($A->slice([0,4,2], [0,5,2]));
# A[0:4:2, 0:5:2]
```

```
# [[ 0  2  4]
#  [10 12 14]]
```

```
[[ 0  1  2  3  4]
 [10 11 12 13 14]]
```

```
<AI::MXNet::NDArray 2x5 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

```
[[ 0  2  4]
 [ 5  7  9]
 [10 12 14]
 [15 17 19]]
```

```
<AI::MXNet::NDArray 4x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

```
[[ 0  2  4]
 [10 12 14]]
```

```
<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[91]: 1

```
In [92]: #
# 7. begin/end/step Multi-Dimensional Tests
#
nd->print(
  $A->slice(
    begin => [1,1],
    end   => [4,5]
  )
);
# A[1:4,1:5]
# [[ 6  7  8  9]
#  [11 12 13 14]
#  [16 17 18 19]]

nd->print(
  $A->slice(
    begin => [1,1],
    end   => [4,5],
    step  => [2,2]
  )
);
# A[1:4:2,1:5:2]
# [[ 6  8]
#  [16 18]]

nd->print(
  $A->slice(
    begin => undef,
    end   => [4,4],
    step  => [1,2]
  )
);
# A[:4:1,:4:2]
# [[ 0  2]
#  [ 5  7]
#  [10 12]
#  [15 17]]

nd->print(
  $A->slice(
    begin => [2,2],
    end   => undef,
    step  => 2
  )
);
# A[2::2,2::2]
# [[12 14]]
```

```

[[ 6  7  8  9]
 [11 12 13 14]
 [16 17 18 19]]
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[ 6  8]
 [16 18]]
<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[ 0  2]
 [ 5  7]
 [10 12]
 [15 17]]
<AI::MXNet::NDArray 4x2 @cpu(0)> AI::MXNet::NDArray::Slice float32
[[12 14]]
<AI::MXNet::NDArray 1x2 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[92]: 1

```

In [93]: #
# 8. 3D Tests
#
$A = nd->arange(60)->reshape([3,4,5]);

nd->print($A->slice('...',0));
# A[... ,0] # shape => (3,4)

```

```

[[ 0  5 10 15]
 [20 25 30 35]
 [40 45 50 55]]
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[93]: 1

```

In [94]: nd->print($A->slice(0,'...'));
# A[0,...]
# shape => (4,5)

```

```

[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
<AI::MXNet::NDArray 4x5 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[94]: 1

```

In [95]: nd->print($A->slice('...',[0,5,2]));
# A[... ,0:5:2]
# shape => (3,4,3)

```

```

[[[ 0  2  4]
  [ 5  7  9]
  [10 12 14]
  [15 17 19]]

 [[20 22 24]
  [25 27 29]
  [30 32 34]
  [35 37 39]]

 [[40 42 44]
  [45 47 49]
  [50 52 54]
  [55 57 59]]]
<AI::MXNet::NDArray 3x4x3 @cpu(0)> AI::MXNet::NDArray::Slice float32

```

Out[95]: 1

```

In [96]: nd->print($A->slice([1,3],':',[0,5,2]));
# A[1:3,:,0:5:2]

```

```
# shape => (2,4,3)
```

```
[[[20 22 24]
  [25 27 29]
  [30 32 34]
  [35 37 39]]
```

```
[[40 42 44]
 [45 47 49]
 [50 52 54]
 [55 57 59]]]
```

```
<AI::MXNet::NDArray 2x4x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[96]: 1

```
In [97]: nd->print(
          $A->slice(
            begin => [0,1,0],
            end   => [3,4,5],
            step  => [1,2,2]
          )
        );
# A[0:3:1,1:4:2,0:5:2]
# shape => (3,2,3)
```

```
[[[ 5  7  9]
  [15 17 19]]
```

```
[[25 27 29]
 [35 37 39]]
```

```
[[45 47 49]
 [55 57 59]]]
```

```
<AI::MXNet::NDArray 3x2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[97]: 1

```
In [98]: # 9. Ellipsis Tests
$A = nd->arange(24)->reshape([2,3,4]);
nd->print($A->slice('...')); # Entire tensor A[...]
```

```
[[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]
```

```
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]]
```

```
<AI::MXNet::NDArray 2x3x4 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[98]: 1

```
In [99]: nd->print($A->slice('...',0)); # Last coordinate fixed A[...,0] [[ 0,  4,  8], [12, 16, 20]]
```

```
[[ 0  4  8]
 [12 16 20]]
```

```
<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[99]: 1

```
In [100]: nd->print($A->slice(0,'...')); # First coordinate fixed A[0,...] [[0, 1, 2, 3],[4, 5, 6, 7],[8, 9, 10, 11]]
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray::Slice float32
```

Out[100]: 1

```

In [101... nd->print($A->slice(':', '...', 0)); # Middle ellipsis A[:, :, 0] [[ 0,  4,  8], [12, 16,
[[ 0  4  8]
[12 16 20]]
<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
Out[101]: 1

In [102... nd->print($A->slice([0,2], '...')); # Explicit expansion check A[0:2,...] Entire tenso
[[[ 0  1  2  3]
[ 4  5  6  7]
[ 8  9 10 11]]

[[12 13 14 15]
[16 17 18 19]
[20 21 22 23]]]
<AI::MXNet::NDArray 2x3x4 @cpu(0)> AI::MXNet::NDArray::Slice float32
Out[102]: 1

In [103... # 5. Stride Parsing Tests
$A = nd->arange(10);
nd->print($A->slice([1,8,2])); # 1-D A[1:8:2] [1 3 5 7] shape:[4]
[1 3 5 7]
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray::Slice float32
Out[103]: 1

In [104... $A = nd->arange(20)->reshape([4,5]);
nd->print($A->slice([0,4,2], ':')); # 2-D A[0:4:2,:] Expected rows 0 2: [[ 0  1  2  3
[[ 0  1  2  3  4]
[10 11 12 13 14]]
<AI::MXNet::NDArray 2x5 @cpu(0)> AI::MXNet::NDArray::Slice float32
Out[104]: 1

In [105... nd->print($A->slice(':', [0,5,2])); # A[:,0:5:2] Expected columns: 0 2 4: [[ 0  2  4],
[[ 0  2  4]
[ 5  7  9]
[10 12 14]
[15 17 19]]
<AI::MXNet::NDArray 4x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
Out[105]: 1

In [106... nd->print($A->slice([0,4,2], [0,5,2])); # Double stride A[0:4:2,0:5:2]: [[ 0  2  4],[1
[[ 0  2  4]
[10 12 14]]
<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray::Slice float32
Out[106]: 1

In [107... $A = nd->arange(24)->reshape([2,3,4]);
nd->print($A->slice(':', ':', [0,4,2])); # 3-D A[:, :, 0:4:2] [[[ 0  2],[ 4  6],[ 8 10]
[[[ 0  2]
[ 4  6]
[ 8 10]]

[[12 14]
[16 18]
[20 22]]]
<AI::MXNet::NDArray 2x3x2 @cpu(0)> AI::MXNet::NDArray::Slice float32
Out[107]: 1

```

```
In [108... # 6. Error Tests
# nd->print($A->slice('...','...')); # Only one ellipsis (...) is allowed.
# $A = nd->arange(20)->reshape([4,5]);
# nd->print($A->slice(':',':',':',':')); # Too many dimensions on a 2-D tensor.
# nd->print($A->slice([5,3])); # Invalid range: begin > end
# nd->print($A->slice([0,20])); # Out of bounds:
```

2.1.2. Operations

This book is not about software engineering. Our interests are not limited to simply reading and writing data from/to arrays. We want to perform mathematical operations on those arrays. Some of the simplest and most useful operations are the elementwise operations. These apply a standard scalar operation to each element of an array. For functions that take two arrays as inputs, elementwise operations apply some standard binary operator on each pair of corresponding elements from the two arrays. We can create an elementwise function from any function that maps from a scalar to a scalar.

The common standard arithmetic operators (+, -, *, /, and **) have all been lifted to elementwise operations for any identically-shaped tensors of arbitrary shape. We can call elementwise operations on any two tensors of the same shape. In the following example, we use commas to formulate a 5-element tuple, where each element is the result of an elementwise operation.

2.1.2.1. Operations

The common standard arithmetic operators (+, -, *, /, and **) have all been lifted to elementwise operations.

```
In [109... my $x = nd->array([1, 2, 4, 8]);
my $y = nd->array([2, 2, 2, 2]);
```

```
Out[109]: <AI::MXNet::NDArray 4 @cpu(0)>
```

```
In [110... my $z = $x + $y;
print $z->asstr;
```

```
[ 3  4  6 10]
```

```
Out[110]: 1
```

```
In [111... $z = $x - $y;
print $z->asstr;
```

```
[-1  0  2  6]
```

```
Out[111]: 1
```

```
In [112... $z = $x * $y;
print $z->asstr;
```

```
[ 2  4  8 16]
```

```
Out[112]: 1
```

```
In [113... $z = $x / $y;
print $z->asstr;
```

```
[0.5000      1      2      4]
```

```
Out[113]: 1
```

```
In [114... $z = $x ** $y; # The ** operator is exponentiation
print $z->asstr;
```

```
[ 1  4 16 64]
```

```
Out[114]: 1
```

```
In [115... $y = nd->array([2]);
print "x->shape:", dump $x->shape;
print "\ny->shape:", dump $y->shape;
```

```
x->shape:[4]
```

```
y->shape:[1]
```

```
Out[115]: 1
```

```
In [116... $z = $x + $y; # Broadcast fue posible
print $z->asstr;
```

```
[ 3  4  6 10]
```

```
Out[116]: 1
```

```
In [117... $y = nd->array([2, 2]);
print "x->shape:", dump $x->shape;
print "\ny->shape:", dump $y->shape;
```

```
x->shape:[4]
```

```
y->shape:[2]
```

```
Out[117]: 1
```

```
In [118... # $z = $x + $y; # Broadcast no es posible
#print $z->asstr;
# MXNetError: Check failed: l == 1 || r == 1:
# operands could not be broadcast together with shapes [4] [2]
```

Many more operations can be applied elementwise, including unary operators like exponentiation.

```
In [119... print nd->exp($x)->asstr;
```

```
[ 2.7183  7.3891 54.5981 2980.9580]
```

```
Out[119]: 1
```

```
In [120... my $exp_nd = nd->exp($x);
```

```
Out[120]: <AI::MXNet::NDArray 4 @cpu(0)>
```

In addition to elementwise computations, we can also perform linear algebra operations, including vector dot products and matrix multiplication. We will explain the crucial bits of linear algebra (with no assumed prior knowledge) in Section 2.3.

We can also concatenate multiple tensors together, stacking them end-to-end to form a larger tensor. We just need to provide a list of tensors and tell the system along which axis to concatenate. The example below shows what happens when we concatenate two matrices along rows (axis 0, the first element of the shape) vs. columns (axis 1, the second element of the shape). We can see that the first output tensor's axis-0 length (3+3) is the sum of the two input tensors' axis-0 lengths (6); while the second output tensor's axis-1 length (8) is the sum of the two input tensors' axis-1 lengths (4+4).

```
In [121... $x = nd->arange(12)->reshape([3, 4]);
print $x->aspd1;
```

```
[
  [ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]
]
```

Out[121]: 1

```
In [122... $y = nd->array([[2, 1, 4, 3], [1, 2, 3, 4]]);
```

Out[122]: <AI::MXNet::NDArray 2x4 @cpu(0)>

```
In [123... $z = nd->concat($x, $y, dim => 0);
print $z->aspdL;
```

```
[
  [ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]
  [ 2  1  4  3]
  [ 1  2  3  4]
]
```

Out[123]: 1

```
In [124... my @lista2 = ($x, $y, $y, $y);
```

Out[124]: <AI::MXNet::NDArray 3x4 @cpu(0)><AI::MXNet::NDArray 2x4 @cpu(0)><AI::MXNet::NDArray 2x4 @cpu(0)><AI::MXNet::NDArray 2x4 @cpu(0)>

```
In [125... $z = nd->concat(@lista2, dim => 0);
print $z->aspdL;
```

```
[
  [ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]
  [ 2  1  4  3]
  [ 1  2  3  4]
  [ 2  1  4  3]
  [ 1  2  3  4]
  [ 2  1  4  3]
  [ 1  2  3  4]
]
```

Out[125]: 1

```
In [126... print "x->shape:", dump $x->shape;
print "\ny->shape:", dump $y->shape;
```

```
x->shape:[3, 4]
y->shape:[2, 4]
```

Out[126]: 1

```
In [127... # $z = nd->concat($x, $y, dim => 1);
# print $z->asstr;
# MXNetError: Check failed: shape_assign(&(*in_shape)[i], dshape):
# Incompatible input shape: expected [3,-1], got [2,4]
```

```
In [128... print $x->aspdL;
print $y->aspdL;
```

```
[
 [ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
]

[
 [2 1 4 3]
 [1 2 3 4]
]
```

Out[128]: 1

Note: It is recommended to use the `concat()` function instead of `concatenate()` because the output of `concatenate()` to a variable does not allow writing (`writable => 0`), while `concat` does allow writing (`writable => 1`). Information has also been found that `concatenate()` was deprecated.

Sometimes, we want to construct a binary tensor via logical statements. Take $X == Y$ as an example. For each position, if X and Y are equal at that position, the corresponding entry in the new tensor takes a value of 1, meaning that the logical statement $X == Y$ is true at that position; otherwise that position takes 0.

```
In [129... $x = nd->array([1, 2, 3]);
            $y = nd->array([4, 5, 3]);
            $z = $x == $y;
            print $z->asstr; # Equivalente a un assert
```

```
[0 0 1]
```

Out[129]: 1

Summing all the elements in the tensor yields a tensor with only one element.

```
In [130... $x = nd->arange(12)->reshape([3, 4]);
            print $x->asstr;
            print $x->sum()->asstr;
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]][66]
```

Out[130]: 1

```
In [131... print $x->sum(axis => 1)->asstr;
```

```
[ 6 22 38]
```

Out[131]: 1

```
In [132... # Preserves the dimension of original tensor
            print $x->sum(axis => 0, keepdims=>1)->asstr;
```

```
[[12 15 18 21]]
```

Out[132]: 1

```
In [133... print $x->sum(axis => 1)->asstr;
```

```
[ 6 22 38]
```

Out[133]: 1

```
In [134... print $x;
            print $x->asstr;
```

```
<AI::MXNet::NDArray 3x4 @cpu(0)>[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Out[134]: 1

```
In [135... $y = $x->expand_dims(2);
print $y;
print $y->asstr;
```

```
<AI::MXNet::NDArray 3x4x1 @cpu(0)>[[[ 0]
 [ 1]
 [ 2]
 [ 3]]

 [[ 4]
 [ 5]
 [ 6]
 [ 7]]

 [[ 8]
 [ 9]
 [10]
 [11]]]
```

Out[135]: 1

```
In [136... print $y->squeeze(2);
```

```
<AI::MXNet::NDArray 3x4 @cpu(0)>
```

Out[136]: 1

```
In [137... print $x->sum->asscalar, "\n";
print ref($x), "\n";
print ref($x->sum->asscalar);
```

```
66
AI::MXNet::NDArray
SCALAR
```

Out[137]: 1

```
In [138... $x = nd->array([2, 3, 4]);
$y = nd->array([1, 5, 2]);
print $x->maximum($y)->asstr;
#https://mxnet.apache.org/versions/master/api/python/docs/api/np/generated/mxnet.nd.m
```

```
[2 5 4]
```

Out[138]: 1

```
In [139... $x = nd->array([-2, -3, 4, 5]);
print $x->maximum(0)->asstr;
```

```
[0 0 4 5]
```

Out[139]: 1

2.1.2.2. Additional operations

```
In [140... # Power
$x = nd->arange(4);
print $x->power(2)->asstr;
# Instead of nd->power($x, 2);
```

```
[0 1 4 9]
```

Out[140]: 1

```
In [141... # Shuffle
$x = nd->arange(15)->reshape([5, 3]);
print "\nNot shuffled: ", $x->asstr;
print "\nShuffled:      ", $x->shuffle->asstr;
```

```
Not shuffled: [[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]
 [12 13 14]]
Shuffled:      [[ 0  1  2]
 [ 3  4  5]
 [ 9 10 11]
 [12 13 14]
 [ 6  7  8]]
```

Out[141]: 1

```
In [142... sub factorial{
    my $n = shift;
    return $n == 0 ? 1 : nd->prod(nd->arange(start => 1, stop => $n + 1))->asscalar;
}

print factorial(5);
```

120

Out[142]: 1

nd->pick

Picks elements from an input array according to the input indices along the given axis.

Given an input array of shape (d0, d1) and indices of shape (i0,) the result will be an output array of shape (i0,) with:

<https://mxnet.apache.org/versions/1.6/api/r/docs/api/mx.nd.pick.html>

```
In [143... $x = nd->arange(12)->reshape([3,4]);
print $x;
print $x->asstr;
```

```
<AI::MXNet::NDArray 3x4 @cpu(0)>[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Out[143]: 1

```
In [144... # Considere tomar un elemento de cada fila de la matriz x usando la función pick().
# Debe armar un tensor de índices que contenga un solo índice por columna (eje 1).
printf "[1, 2, 3]: %s\n", nd->pick($x, nd->array([1, 2, 3]), axis => 1)->asstr;
printf "[3, 3, 3]: %s\n", nd->pick($x, nd->array([3, 3, 3]), axis => 1)->asstr;
printf "[3, 2, 1]: %s\n", nd->pick($x, nd->array([3, 2, 1]), axis => 1)->asstr;
```

```
[1, 2, 3]: [ 1  6 11]
[3, 3, 3]: [ 3  7 11]
[3, 2, 1]: [3 6 9]
```

Out[144]: 1

```
In [145... $x = nd->array([[ 1,  2], [ 3,  4], [ 5,  6]]);
print $x;
print $x->asstr;
```

```
<AI::MXNet::NDArray 3x2 @cpu(0)>[[1 2]
[3 4]
[5 6]]
```

Out[145]: 1

```
In [146... # picks elements with specified indices along axis 0
print nd->pick($x, nd->array([1, 2]), axis => 0)->asstr;
# = [ 3,  6]
```

[3 6]

Out[146]: 1

```
In [147... # picks elements with specified indices along axis 1
print nd->pick($x, nd->array([1, 0, 1]), axis => 1)->asstr;
# [2, 3, 6]
```

[2 3 6]

Out[147]: 1

nd->take

Takes elements from an input array along the given axis.

This function slices the input array along a particular axis with the provided indices.

Given data tensor of rank $r \geq 1$, and indices tensor of rank q , gather entries of the axis dimension of data (by default outer-most one as $\text{axis}=0$) indexed by indices, and concatenates them in an output tensor of rank $q + (r - 1)$. <https://mxnet.apache.org/versions/1.6/api/r/docs/api/mx.nd.take.html>

```
In [148... print $x->asstr;
print nd->take($x, nd->array([2, 0, 1, 1]), axis => 0)->asstr;
```

```
[[1 2]
[3 4]
[5 6]][[5 6]
[1 2]
[3 4]
[3 4]]
```

Out[148]: 1

```
In [149... print $x->asstr;
print nd->take($x, nd->array([1, 0]), axis => 1)->asstr;
```

```
[[1 2]
[3 4]
[5 6]][[2 1]
[4 3]
[6 5]]
```

Out[149]: 1

nd->contrib->arange_like

Return an array with evenly spaced values. If axis is not given, the output will have the same shape as the input array. Otherwise, the output will be a 1-D array with size of the specified axis in input shape.

```
In [150... $y = nd->array([[0.14883883, 0.7772398, 0.94865847, 0.7225052 ],
                  [0.23729339, 0.6112595, 0.66538996, 0.5132841 ],
                  [0.30822644, 0.9912457, 0.15502319, 0.7043658 ]]);
```

```
print nd->contrib->arange_like($y)->asstr;
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Out[150]: 1

```
In [151... printf "Shape as NDAarray: %s\n", ref $x->shape_array();
printf "Shape: %s\n", $x->shape_array()->asstr;
```

```
Shape as NDAarray: AI::MXNet::NDAarray
Shape: [3 2]
```

Out[151]: 1

```
In [152... printf "Shape as Perl ARRAY: %s\n", ref $x->shape;
printf "Shape: %s\n", dump $x->shape; # [3 4]
```

```
Shape as Perl ARRAY: ARRAY
Shape: [3, 2]
```

Out[152]: 1

nd->flatten

Flattens the input array into a 2-D array by collapsing the higher dimensions.

For an input array with shape (d1, d2, ..., dk), flatten operation reshapes the input array into an output array of shape (d1, d2...dk). Note that the behavior of this function is different from `numpy.ndarray.flatten`, which behaves similar to `mxnet.ndarray.reshape((-1,))`.

<https://mxnet.apache.org/versions/1.6/api/r/docs/api/mx.nd.flatten.html>

```
In [153... $x = nd->array([[ [1,2,3], [4,5,6], [7,8,9]], [[1,2,3], [4,5,6], [7,8,9] ]]);
```

Out[153]: <AI::MXNet::NDAarray 2x3x3 @cpu(0)>

```
In [154... nd->flatten($x)->asstr;
# [[ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.],
# [ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.]]
```

```
Out[154]: [[1 2 3 4 5 6 7 8 9]
 [1 2 3 4 5 6 7 8 9]]
```

```
In [155... nd->flatten(nd->array([[1, 2]]))->asstr;
```

Out[155]: [[1 2]]

```
In [156... $x = nd->array([[
[1,2,3],
[4,5,6],
[7,8,9]
],
[ [1,2,3],
[4,5,6],
[7,8,9]
]]);
```

Out[156]: <AI::MXNet::NDAarray 2x3x3 @cpu(0)>

```
In [157... # Flattens the input array into a 2-D array by collapsing the higher dimensions.
my $y = nd->flatten($x);
```

```
# Note that the behavior of this function is different from numpy.ndarray.flatten,
# which behaves similar to mxnet.ndarray.reshape((-1,)).
print $y->asstr;
```

```
[[1 2 3 4 5 6 7 8 9]
 [1 2 3 4 5 6 7 8 9]]
```

Out[157]: 1

full()

```
In [158... nd->full([2, 2], 5)->asstr;
```

```
Out[158]: [[5 5]
 [5 5]]
```

```
In [159... $var = nd->full([2, 3, 4], 10);
print $var->asstr;
```

```
[[10 10 10 10]
 [10 10 10 10]
 [10 10 10 10]]
```

```
[[10 10 10 10]
 [10 10 10 10]
 [10 10 10 10]]]
```

Out[159]: 1

nd->swapaxes

<https://mxnet.apache.org/versions/master/api/python/docs/api/np/generated/mxnet.np.swapaxes.html>

swapaxes(a, axis1, axis2) Interchange two axes of an array.

Parameters a (ndarray) – Input array.

axis1 (int) – First axis.

axis2 (int) – Second axis.

Returns a_swapped – Swapped array. This is always a copy of the input array.

Return type ndarray

```
In [160... $x = nd->array([[1, 2, 3]]);
print nd->swapaxes($x, 0, 1)->asstr;
# array([[1.],
#        [2.],
#        [3.]])
```

```
[[1]
 [2]
 [3]]
```

Out[160]: 1

```
In [161... $x = nd->array([[[0,1], [2,3]], [[4,5], [6,7]]]);
print nd->swapaxes($x, 0, 2)->asstr;
# array([[[0., 4.],
#        [2., 6.]],
#        [[1., 5.],
#        [3., 7.]])
```

```
[[[0 4]
   [2 6]]

 [[1 5]
  [3 7]]]
```

Out[161]: 1

nd->std

Computes the standard deviation of the input NDAarray.

Similar function in the MXNet ndarray as `numpy.std`.

See Also

<https://numpy.org/doc/stable/reference/generated/numpy.std.html>.

Parameters

`$x` : NDAarray

Input array.

`:$axis=` : Int or ArrayRef[Int], optional

Axis or axes along which the standard deviation is computed.

For example:

`axis => 0`

`axis => 1`

`axis => [0, 1]`

By default, the standard deviation is computed over all elements of the array.

`:$keepdims=0` : Bool, optional

If true, the reduced axes are left in the result as dimensions with size one, allowing the result to broadcast correctly against the input array.

Returns

`$out` : NDAarray

The standard deviation of the input array.

Notes

The standard deviation is computed as::

$$\sqrt{\text{sum}((x - \text{mean}(x)) ** 2) / N}$$

where N is the number of elements along the reduction axis.

This corresponds to `numpy.std(..., ddof=0)`.

```
In [162... my $x = nd->arange(24)->reshape([2,3,4]);
nd->print($x);
```

```
[[[ 0  1  2  3]
   [ 4  5  6  7]
   [ 8  9 10 11]]
```

```
[[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]]
```

```
<AI::MXNet::NDArray 2x3x4 @cpu(0)> AI::MXNet::NDArray float32
```

Out[162]: 1

In [163... `# axis=None`

```
nd->print(nd->std($x));
# 6.922186552431729
```

6.9222

```
<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float32
```

Out[163]: 1

In [164... `print nd->_mpi_std($x)->asstr;`

6.9222

Out[164]: 1

In [165... `# axis=0`

```
nd->print(nd->std($x, axis=>0));
# [[6. 6. 6. 6.]
#  [6. 6. 6. 6.]
#  [6. 6. 6. 6.]]
```

```
[[6 6 6 6]
 [6 6 6 6]
 [6 6 6 6]]
```

```
<AI::MXNet::NDArray 3x4 @cpu(0)> AI::MXNet::NDArray float32
```

Out[165]: 1

In [166... `print nd->_mpi_std($x, axis=>0)->asstr;`

```
[[6 6 6 6]
 [6 6 6 6]
 [6 6 6 6]]
```

Out[166]: 1

In [167... `# axis=1`

```
nd->print(nd->std($x, axis=>1));
# [[3.26598632 3.26598632 3.26598632 3.26598632]
#  [3.26598632 3.26598632 3.26598632 3.26598632]]
```

```
[[3.2660 3.2660 3.2660 3.2660]
 [3.2660 3.2660 3.2660 3.2660]]
```

```
<AI::MXNet::NDArray 2x4 @cpu(0)> AI::MXNet::NDArray float32
```

Out[167]: 1

In [168... `print nd->_mpi_std($x, axis=>1)->asstr;`

```
[[3.2660 3.2660 3.2660 3.2660]
 [3.2660 3.2660 3.2660 3.2660]]
```

Out[168]: 1

In [169... `# axis=2`

```
nd->print(nd->std($x, axis=>2));
```

```
# [[1.11803399 1.11803399 1.11803399]]  
# [1.11803399 1.11803399 1.11803399]]
```

```
[[1.1180 1.1180 1.1180]  
 [1.1180 1.1180 1.1180]]
```

```
<AI::MXNet::NDArray 2x3 @cpu(0)> AI::MXNet::NDArray float32
```

Out[169]: 1

```
In [170... print nd->_mpi_std($x, axis=>2)->asstr;
```

```
[[1.1180 1.1180 1.1180]  
 [1.1180 1.1180 1.1180]]
```

Out[170]: 1

```
In [171... # axis=[0,1]
```

```
nd->print(nd->std($x, axis=>[0,1]));  
# [6.83130051 6.83130051 6.83130051 6.83130051]
```

```
[6.8313 6.8313 6.8313 6.8313]
```

```
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32
```

Out[171]: 1

```
In [172... print nd->_mpi_std($x, axis=>[0,1])->asstr;
```

```
[6.8313 6.8313 6.8313 6.8313]
```

Out[172]: 1

```
In [173... # axis=[1,2]
```

```
nd->print(nd->std($x, axis=>[1,2]));  
# [3.45205253 3.45205253]
```

```
[3.4521 3.4521]
```

```
<AI::MXNet::NDArray 2 @cpu(0)> AI::MXNet::NDArray float32
```

Out[173]: 1

```
In [174... # axis=[0,2]
```

```
nd->print(nd->std($x, axis=>[0,2]));  
# [6.10327781 6.10327781 6.10327781]
```

```
[6.1033 6.1033 6.1033]
```

```
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray float32
```

Out[174]: 1

```
In [175... print nd->_mpi_std($x, axis=>[0,2])->asstr;
```

```
[6.1033 6.1033 6.1033]
```

Out[175]: 1

```
In [176... # keepdims=True
```

```
nd->print(nd->std($x, axis=>[0,2], keepdims=>1));  
# [[6.10327781]  
#   [6.10327781]  
#   [6.10327781]]
```

```
[[[6.1033]  
   [6.1033]  
   [6.1033]]]
```

```
<AI::MXNet::NDArray 1x3x1 @cpu(0)> AI::MXNet::NDArray float32
```

Out[176]: 1

```
In [177... print nd->_mpi_std($x, axis=>[0,2], keepdims=>1)->asstr;
```

```
[[[6.1033]  
  [6.1033]  
  [6.1033]]]
```

```
Out[177]: 1
```

nd->cov

Estimate a covariance matrix, given data and optional weights.

<https://numpy.org/doc/stable/reference/generated/numpy.cov.html>

Covariance indicates the level to which two variables vary together. If we examine N-dimensional samples, $X = [x_1, x_2, \dots, x_N]^T$, then the covariance matrix element $C(i,j)$ is the covariance of x_i and x_j . The element $C(i,i)$ is the variance of x_i .

Similar function in the MXNet ndarray as `numpy.cov`.

See Also

<https://numpy.org/doc/stable/reference/generated/numpy.cov.html>.

Parameters

`$x` : NDArray

A 1-D or 2-D NDArray containing variables and observations. Each row of `$x` represents a variable, and each column a single observation of all those variables. See also `:$rowvar` below.

`:$y=` : NDArray, optional

An additional set of variables and observations. `$y` must have the same form as that of `$x`.

`:$rowvar=1` : Bool, optional

If true (default), each row represents a variable, with observations stored in the columns.

If false, each column represents a variable and the rows contain observations.

`:$bias=0` : Bool, optional

Default normalization is by $(N - 1)$, where N is the number of observations (unbiased estimate).

If true, normalization is by N .

This value may be overridden by `:$ddof`.

`:$ddof=` : Int, optional

Overrides the default normalization implied by `:$bias`.

For example:

```
ddof => 0    # normalize by N  
ddof => 1    # normalize by N - 1
```

`:$fweights=` : NDArray, optional

A 1-D NDArray of integer frequency weights.

Each element specifies how many times the corresponding observation vector should be repeated.

`:$weights=` : NDAarray, optional
A 1-D NDAarray of observation weights.

Larger values indicate observations that should contribute more strongly to the covariance estimate.

When used with:

`ddof => 0`

the weights may be interpreted as probabilities.

`:$dtype=` : Str, optional
Data type used internally for the computation.

Examples:

`dtype => 'float32'`
`dtype => 'float64'`

Returns

`$out` : NDAarray
Covariance matrix of the variables.

If the input contains a single variable, a scalar NDAarray containing the variance is returned.

Notes

The covariance matrix is computed as::

$$C = (X - \text{mean}(X)) (X - \text{mean}(X))^T / \text{fact}$$

where the normalization factor depends on the values of `:$bias`, `:$ddof`, `:$fweights` and `:$aweights`.

This implementation follows the behavior of `numpy.cov`, including weighted covariance estimation using frequency and analytical weights.

```
In [178... # Test 0: Caso escalar
```

```
$x = nd->array([5]);  
nd->print(  
  nd->cov($x)  
);
```

NaN

<AI::MXNet::NDAarray @cpu(0)> AI::MXNet::NDAarray float32

Out[178]: 1

```
In [179... # Test 1: vector 1D
```

```
my $x = nd->array([1,2,3]);  
  
# print nd->cov($x)->asscalar;  
nd->print(  
  nd->cov($x)
```

```
);

$x = nd->array([1,3,5]);
nd->print(
  nd->cov($x)
);
```

1.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

4.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

Out[179]: 1

In [180... *# Test 2: vector con varianza distinta de 1*

```
$x = nd->array([1,3,5]);
nd->print(nd->cov($x));
```

4.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

Out[180]: 1

In [181... *# Test 3: matriz 2x3*

```
$x = nd->array([
  [1,2,3],
  [4,5,6]
]);

nd->print(
  nd->cov($x)
);
```

[[1 1]

[1 1]]

<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64

Out[181]: 1

In [182... *# Test 4: rowvar=False*

```
$x = nd->array([
  [1,4],
  [2,5],
  [3,6]
]);

nd->print(
  nd->cov($x,rowvar=>0)
);
```

[[1 1]

[1 1]]

<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64

Out[182]: 1

In [183... *# Test 5: x,y 1D*

```
$x = nd->array([1,2,3]);
my $y = nd->array([4,5,6]);

nd->print(
  nd->cov($x, $y)
);
```

```
[[1 1]
 [1 1]]
<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64
```

Out[183]: 1

In [184... *# Test 6: correlación negativa*

```
$x = nd->array([1,2,3]);
$y = nd->array([3,2,1]);

nd->print(
    nd->cov($x, $y)
);
```

```
[[ 1 -1]
 [-1  1]]
<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64
```

Out[184]: 1

In [185... *# Test 7: tres variables*

```
$x = nd->array([
    [1,2,3],
    [4,5,6],
    [7,8,9]
]);

nd->print(
    nd->cov($x)
);
```

```
[[1 1 1]
 [1 1 1]
 [1 1 1]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float64
```

Out[185]: 1

In [186... *# Test 8: comparación exhaustiva*

```
$x = nd->arange(12)->reshape([3,4]);

print "rowvar=1\n";
nd->print(
    nd->cov($x)
);

print "rowvar=0\n";
nd->print(
    nd->cov($x, rowvar=>0)
);
```

```
rowvar=1
[[1.6667 1.6667 1.6667]
 [1.6667 1.6667 1.6667]
 [1.6667 1.6667 1.6667]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float64
rowvar=0
[[16 16 16 16]
 [16 16 16 16]
 [16 16 16 16]
 [16 16 16 16]]
<AI::MXNet::NDArray 4x4 @cpu(0)> AI::MXNet::NDArray float64
```

Out[186]: 1

In [187]... *# Test 9: ddof*

```
my $x = nd->array([1,2,3]);

nd->print(nd->cov($x));
nd->print(nd->cov($x, bias=>1));
nd->print(nd->cov($x, ddof=>0));
nd->print(nd->cov($x, ddof=>1));
nd->print(nd->cov($x, ddof=>2));
```

1.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

0.6667

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

0.6667

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

1.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

2.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

Out[187]: 1

In [188]... *# Test 10: fweights*

```
my $x = nd->array([1,2,3]);
my $f = nd->array([1,2,1]);

nd->print(
  nd->cov($x, fweights=>$f)
);
# 0.6666666666666666
```

0.6667

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

Out[188]: 1

In [189]... *# Test 11: aweights*

```
my $x = nd->array([1,2,3]);
my $w = nd->array([0.2,0.5,0.3]);

nd->print(
  nd->cov($x, aweights=>$w)
);
# 0.7903225806451614
```

0.7903

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

Out[189]: 1

In [190]... *# Tests 12: fweights + aweights*

```
my $x = nd->array([1,2,3]);

nd->print(
  nd->cov(
    $x,
    fweights=>nd->array([1,2,1]),
    aweights=>nd->array([1,0.5,2])
  )
);
# 1.0476190476190474
```

1.0476

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

Out[190]: 1

```
In [191]... my $x = nd->array([1,2,3]);

nd->print(
  nd->cov(
    $x,
    aweights=>nd->array([1,1,1])
  )
);

nd->print(
  nd->cov(
    $x,
    fweights=>nd->array([1,1,1])
  )
);
# 1.0
```

1.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

1.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

Out[191]: 1

```
In [192]... my $x = nd->array([1,2,3]);
my $w = nd->array([0.2,0.5,0.3]);

nd->print(nd->cov($x,aweights=>$w,ddof=>0));
nd->print(nd->cov($x,aweights=>$w,ddof=>1));
nd->print(nd->cov($x,aweights=>$w,ddof=>2));
# 0.49
# 0.7903225806451614
# 2.0416666666666667
```

0.4900

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

0.7903

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

2.0417

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

Out[192]: 1

```
In [193]... # Test 13: Tipo

my $x = nd->array([1,2,3]);

nd->print(
  nd->cov($x, dtype=>'float32')
);
```

1.0000

<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float32

Out[193]: 1

```
In [194]... # Test 14: Matrices ponderadas

my $x = nd->array([
  [1,2,3],
  [4,5,6]
]);
```

```

my $w = nd->array([1,2,1]);

nd->print(
  nd->cov(
    $x,
    fweights=>$w
  )
);
# Este caso ejerce simultáneamente:

# broadcasting de pesos
# media ponderada
# dot ponderado
# normalización ponderada

# y es una excelente prueba de regresión.
# [[0.66666667 0.66666667]
#   [0.66666667 0.66666667]]

```

```

[[0.6667 0.6667]
 [0.6667 0.6667]]
<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64

```

Out[194]: 1

In [195... # Test 15: Caso especial: Inf

```

my $x = nd->array([1,2,3]);
my $w = nd->array([1,1,1]);

nd->print(
  nd->cov(
    $x,
    aweights=> $w,
    ddof=>3)
);

```

```

Inf
<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

```

Out[195]: 1

Degrees of freedom <= 0 for slice.

```

In [196... my $x = nd->array([1,2,3]);
my $w = nd->array([1,1,1]);

nd->print(nd->cov($x, aweights=>$w, ddof=>2)); # 2
nd->print(nd->cov($x, aweights=>$w, ddof=>3)); # Inf
nd->print(nd->cov($x, aweights=>$w, ddof=>4)); # -2

```

```

2.0000
<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64
Inf
<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64
-2.0000
<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float64

```

Out[196]: 1

Degrees of freedom <= 0 for slice.

Degrees of freedom <= 0 for slice.

In [197... # Test 16: Caso especial: X.shape[0] == 0

```

my $x = nd->empty([0, 5]);

nd->print($x);

```

```
nd->print(
    nd->cov($x)
);
```

```
[]
<AI::MXNet::NDArray 0x5 @cpu(0)> AI::MXNet::NDArray float32
>[]
<AI::MXNet::NDArray 0x0 @cpu(0)> AI::MXNet::NDArray float64
```

Out[197]: 1

nd->corrcoef

Computes the Pearson product-moment correlation coefficient matrix.

Similar function in the MXNet ndarray as `numpy.corrcoef`.

See Also

<https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html>
<https://mxnet.apache.org/versions/master/api/python/docs/api/np/generated/mx>
<https://numpy.org/doc/stable/reference/generated/numpy.corrcoef.html>
https://en.wikipedia.org/wiki/Pearson_correlation_coefficient
<https://www.howtoexcel.org/correlation-coefficient/>

The relationship between the correlation coefficient matrix, $C<R>$, and the covariance matrix, $C<C>$, is

$$R[i,j] = C[i,j] / \sqrt{C[i,i] * C[j,j]}$$

The values of the correlation coefficients are between -1 and 1, inclusive.

Parameters

`$x` : NDArray

A 1-D or 2-D array containing variables and observations.
 By default, each row represents a variable and each column represents an observation.

`:$y=` : NDArray, optional

An additional set of variables and observations.
 $C<$y>$ must have the same number of observations as $C<$x>$.

`:$rowvar=1` : Bool, optional

If true (default), each row represents a variable and each column an observation. Otherwise, the relationship is transposed so that each column represents a variable.

`:$dtype=` : Dtype, optional

Data type of the result. If specified, the input arrays are converted to this type before computation.

Returns

`$out` : NDArray

Correlation coefficient matrix.

If only one variable is provided, a scalar correlation coefficient is returned.

See Also

`cov` : Covariance matrix.

Notes

This function computes the covariance matrix using `C<cov()>` and then normalizes it by the standard deviations of the variables. Any supported arguments accepted by `C<cov()>`, such as `C<rowvar>`, `C<dtype>`, `C<fweights>`, `C<aweights>`, `C<bias>`, or `C<ddof>`, are forwarded to `C<cov()>`.

Due to floating-point rounding errors, the resulting matrix may not be exactly symmetric, diagonal elements may differ slightly from 1, and correlation coefficients may slightly exceed the interval `[-1, 1]`. Values are clipped to the interval `[-1, 1]` when possible.

Undefined correlations resulting from zero variance variables produce NaN values, consistent with NumPy behaviour.

```
In [198... my $x = nd->array([1,2,3]);
```

```
nd->print(  
    nd->corrcoef($x)  
);  
# 1.0
```

```
1.0000
```

```
<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[198]: 1
```

```
In [199... my $x = nd->array([5]);
```

```
nd->print(  
    nd->corrcoef($x)  
);  
# NaN
```

```
NaN
```

```
<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[199]: 1
```

```
In [200... my $x = nd->array([1,2,3]);  
my $y = nd->array([4,5,6]);
```

```
nd->print(  
    nd->corrcoef($x,y=>$y)  
);  
# [[1. 1.]  
# [1. 1.]]
```

```
[[1 1]
```

```
[1 1]]
```

```
<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64
```

```
Out[200]: 1
```

```
In [201... my $x = nd->array([1,2,3]);  
my $y = nd->array([3,2,1]);
```

```
nd->print(  
    nd->corrcoef($x, y=>$y)  
);  
# [[ 1. -1.]  
# [-1.  1.]]
```

```
[[ 1 -1]
 [-1 1]]
<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64
```

Out[201]: 1

```
In [202... # varianza zero

my $x = nd->array([7,7,7]);

nd->print(
  nd->corrcoef($x)
);
# NaN
```

NaN

```
<AI::MXNet::NDArray @cpu(0)> AI::MXNet::NDArray float32
```

Out[202]: 1

```
In [203... my $x = nd->arange(12)->reshape([3,4]);

nd->print(
  nd->corrcoef($x)
);
# [[1 1 1]
#   [1 1 1]
#   [1 1 1]]
```

```
[[1 1 1]
 [1 1 1]
 [1 1 1]]
```

```
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float64
```

Out[203]: 1

```
In [204... # In this example we take an array, ``x`` and compute the row-wise
# Pearson correlation coefficients because ``rowvar`` is true by default.
```

```
my $x = nd->array([[0.77395605, 0.43887844, 0.85859792],
                   [0.69736803, 0.09417735, 0.97562235],
                   [0.7611397 , 0.78606431, 0.12811363]]);

nd->print(
  nd->corrcoef($x)
);
# [[ 1.          ,  0.99256089, -0.68080986],
#   [ 0.99256089,  1.          , -0.76492172],
#   [-0.68080986, -0.76492172,  1.          ]]
```

```
[[ 1.0000  0.9926 -0.6808]
 [ 0.9926  1.0000 -0.7649]
 [-0.6808 -0.7649  1.0000]]
```

```
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float64
```

Out[204]: 1

```
In [205... # If we add another set of variables and observations ``y``, we can
# compute the row-wise Pearson correlation coefficients between the
# variables in ``x`` and ``y``.
```

```
my $y = nd->array([[0.45038594, 0.37079802, 0.92676499],
                   [0.64386512, 0.82276161, 0.4434142 ],
                   [0.22723872, 0.55458479, 0.06381726]]);

nd->print(
  nd->corrcoef($x, y=>$y)
);
# [[          1  0.992561 -0.68081  0.750082 -0.934284 -0.990041]
#   [0.992561          1 -0.764922  0.82502 -0.970741 -0.999816]
```

```
# [-0.68081 -0.764922 1 -0.995072 0.897214 0.777147]
# [ 0.750082 0.82502 -0.995072 1 -0.936579 -0.835717]
# [-0.934284 -0.970741 0.897214 -0.936579 1 0.975172]
# [-0.990041 -0.999816 0.777147 -0.835717 0.975172 1]]

[[ 1.0000 0.9926 -0.6808 0.7501 -0.9343 -0.9900]
 [ 0.9926 1.0000 -0.7649 0.8250 -0.9707 -0.9998]
 [-0.6808 -0.7649 1.0000 -0.9951 0.8972 0.7771]
 [ 0.7501 0.8250 -0.9951 1.0000 -0.9366 -0.8357]
 [-0.9343 -0.9707 0.8972 -0.9366 1.0000 0.9752]
 [-0.9900 -0.9998 0.7771 -0.8357 0.9752 1.0000]]
<AI::MXNet::NDArray 6x6 @cpu(0)> AI::MXNet::NDArray float64
```

Out[205]: 1

In [206... *# Finally if we use the option ``rowvar=False``, the columns are now
being treated as the variables and we will find the column-wise Pearson
correlation coefficients between variables in ``x`` and ``y``.*

```
nd->print(
  nd->corrcoef($x, y=>$y, rowvar=>0)
);

# [[ 1 0.775981 -0.474585 -0.750786 -0.966555 0.224238]
# [ 0.775981 1 -0.923467 -0.999239 -0.588266 -0.44069]
# [-0.474585 -0.923467 1 0.93773 0.232976 0.751375]
# [-0.750786 -0.999239 0.93773 1 0.556275 0.47537]
# [-0.966555 -0.588266 0.232976 0.556275 1 -0.466665]
# [ 0.224238 -0.44069 0.751375 0.47537 -0.466665 1]]

[[ 1.0000 0.7760 -0.4746 -0.7508 -0.9666 0.2242]
 [ 0.7760 1.0000 -0.9235 -0.9992 -0.5883 -0.4407]
 [-0.4746 -0.9235 1.0000 0.9377 0.2330 0.7514]
 [-0.7508 -0.9992 0.9377 1.0000 0.5563 0.4754]
 [-0.9666 -0.5883 0.2330 0.5563 1.0000 -0.4667]
 [ 0.2242 -0.4407 0.7514 0.4754 -0.4667 1.0000]]
<AI::MXNet::NDArray 6x6 @cpu(0)> AI::MXNet::NDArray float64
```

Out[206]: 1

In [207... **my** \$X = nd->array([
 [1,2,3],
 [4,5,6]
]);

nd->print(
 nd->corrcoef(\$X, rowvar=>1)
);

nd->print(
 nd->corrcoef(\$X, rowvar=>0)
);

```
[[1 1]
 [1 1]]
<AI::MXNet::NDArray 2x2 @cpu(0)> AI::MXNet::NDArray float64
[[1 1 1]
 [1 1 1]
 [1 1 1]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float64
```

Out[207]: 1

Complementary tensor functions now available in MXNet

You can use `nd->list_functions()`; to list all functions natively available in MXNet. All these functions have been integrated with MXNet without the prefixes `_`, `_np_`, `_npi_`, `_npx_`. That means you can now invoke `nd->diff` on the fly which internally invokes `_npi_diff` to fill the gap that `nd->diff` is not originally listed below.

In [208... `nd->list_functions();`

Activation
BatchNorm
BatchNorm_v1
BilinearSampler
BlockGrad
CTCLoss
Cast
Concat
Convolution
Convolution_v1
Correlation
Crop
CuDNNBatchNorm
Custom
Deconvolution
Dropout
ElementWiseSum
Embedding
Flatten
FullyConnected
GridGenerator
GroupNorm
IdentityAttachKLSparseReg
InstanceNorm
L2Normalization
LRN
LayerNorm
LeakyReLU
LinearRegressionOutput
LogisticRegressionOutput
MAERegressionOutput
MakeLoss
Pad
Pooling
Pooling_v1
RNN
ROIPooling
Reshape
SVMOutput
SequenceLast
SequenceMask
SequenceReverse
SliceChannel
Softmax
SoftmaxActivation
SoftmaxOutput
SpatialTransformer
SwapAxis
UpSampling
_CachedOp
_CachedOpThreadSafe
_CrossDeviceCopy
_CustomFunction
_Div
_DivScalar
_Equal
_EqualScalar
_FusedOp
_FusedOpHelper
_FusedOpOutHelper
_Greater
_GreaterEqualScalar
_GreaterScalar
_Greater_Equal
_Hypot
_HypotScalar

`_Lesser`
`_LesserEqualScalar`
`_LesserScalar`
`_Lesser_Equal`
`_LogicalAndScalar`
`_LogicalOrScalar`
`_LogicalXorScalar`
`_Logical_And`
`_Logical_Or`
`_Logical_Xor`
`_Maximum`
`_MaximumScalar`
`_Minimum`
`_MinimumScalar`
`_Minus`
`_MinusScalar`
`_Mod`
`_ModScalar`
`_Mul`
`_MulScalar`
`_NDArray`
`_Native`
`_NoGradient`
`_NotEqualScalar`
`_Not_Equal`
`_Plus`
`_PlusScalar`
`_Power`
`_PowerScalar`
`_RDivScalar`
`_RMinusScalar`
`_RModScalar`
`_RPowerScalar`
`_adamw_update`
`_add`
`_arange`
`_backward_Activation`
`_backward_BatchNorm`
`_backward_BatchNorm_v1`
`_backward_BilinearSampler`
`_backward_CachedOp`
`_backward_Concat`
`_backward_Convolution`
`_backward_Convolution_v1`
`_backward_Correlation`
`_backward_Crop`
`_backward_CuDNNBatchNorm`
`_backward_Custom`
`_backward_CustomFunction`
`_backward_Deconvolution`
`_backward_Dropout`
`_backward_Embedding`
`_backward_FullyConnected`
`_backward_GridGenerator`
`_backward_GroupNorm`
`_backward_IdentityAttachKLSparseReg`
`_backward_InstanceNorm`
`_backward_L2Normalization`
`_backward_LRN`
`_backward_LayerNorm`
`_backward_LeakyReLU`
`_backward_MakeLoss`
`_backward_Pad`
`_backward_Pooling`
`_backward_Pooling_v1`
`_backward_RNN`

_backward_ROIAlign
_backward_ROIPooling
_backward_RR0IAlign
_backward_SVMOutput
_backward_SequenceLast
_backward_SequenceMask
_backward_SequenceReverse
_backward_SliceChannel
_backward_SoftmaxActivation
_backward_SoftmaxOutput
_backward_SparseEmbedding
_backward_SpatialTransformer
_backward_SwapAxis
_backward_UpSampling
_backward__CrossDeviceCopy
_backward__NDArray
_backward__Native
_backward__contrib_DeformableConvolution
_backward__contrib_DeformablePSROIpooling
_backward__contrib_ModulatedDeformableConvolution
_backward__contrib_MultiBoxDetection
_backward__contrib_MultiBoxPrior
_backward__contrib_MultiBoxTarget
_backward__contrib_MultiProposal
_backward__contrib_PSR0IPooling
_backward__contrib_Proposal
_backward__contrib_SyncBatchNorm
_backward__contrib_count_sketch
_backward__contrib_fft
_backward__contrib_ifft
_backward_abs
_backward_add
_backward_amp_cast
_backward_amp_multicast
_backward_arccos
_backward_arccosh
_backward_arcsin
_backward_arcsinh
_backward_arctan
_backward_arctanh
_backward_backward_FullyConnected
_backward_broadcast_add
_backward_broadcast_div
_backward_broadcast_exponential
_backward_broadcast_gumbel
_backward_broadcast_hypot
_backward_broadcast_logistic
_backward_broadcast_maximum
_backward_broadcast_minimum
_backward_broadcast_mod
_backward_broadcast_mul
_backward_broadcast_normal
_backward_broadcast_pareto
_backward_broadcast_power
_backward_broadcast_rayleigh
_backward_broadcast_sub
_backward_broadcast_weibull
_backward_cast
_backward_cbrt
_backward_clip
_backward_col2im
_backward_cond
_backward_contrib_AdaptiveAvgPooling2D
_backward_contrib_BatchNormWithReLU
_backward_contrib_BilinearResize2D
_backward_contrib_bipartite_matching

_backward_contrib_boolean_mask
_backward_contrib_box_iou
_backward_contrib_box_nms
_backward_copy
_backward_cos
_backward_cosh
_backward_ctc_loss
_backward_degrees
_backward_diag
_backward_div
_backward_div_scalar
_backward_dot
_backward_erf
_backward_erfinv
_backward_expml
_backward_foreach
_backward_gamma
_backward_gammaln
_backward_gather_nd
_backward_hard_sigmoid
_backward_hypot
_backward_hypot_scalar
_backward_im2col
_backward_image_crop
_backward_image_normalize
_backward_interleaved_matmul_encdec_qk
_backward_interleaved_matmul_encdec_valatt
_backward_interleaved_matmul_selfatt_qk
_backward_interleaved_matmul_selfatt_valatt
_backward_linalg_det
_backward_linalg_extractdiag
_backward_linalg_extracttrian
_backward_linalg_gelqf
_backward_linalg_gemm
_backward_linalg_gemm2
_backward_linalg_inverse
_backward_linalg_makediag
_backward_linalg_maketrian
_backward_linalg_potrf
_backward_linalg_potri
_backward_linalg_slogdet
_backward_linalg_sumlogdiag
_backward_linalg_syevd
_backward_linalg_syrk
_backward_linalg_trmm
_backward_linalg_trsm
_backward_linear_reg_out
_backward_log
_backward_log10
_backward_log1p
_backward_log2
_backward_log_softmax
_backward_logistic_reg_out
_backward_mae_reg_out
_backward_max
_backward_maximum
_backward_maximum_scalar
_backward_mean
_backward_min
_backward_minimum
_backward_minimum_scalar
_backward_mod
_backward_mod_scalar
_backward_moments
_backward_mul
_backward_mul_scalar

`_backward_nanprod`
`_backward_nansum`
`_backward_norm`
`_backward_np_average`
`_backward_np_broadcast_to`
`_backward_np_column_stack`
`_backward_np_concat`
`_backward_np_cumsum`
`_backward_np_diag`
`_backward_np_diagflat`
`_backward_np_diagonal`
`_backward_np_dot`
`_backward_np_dstack`
`_backward_np_hstack`
`_backward_np_matmul`
`_backward_np_max`
`_backward_np_mean`
`_backward_np_min`
`_backward_np_prod`
`_backward_np_sum`
`_backward_np_trace`
`_backward_np_vstack`
`_backward_np_where`
`_backward_np_where_lscalar`
`_backward_np_where_rscalar`
`_backward_npi_arctan2`
`_backward_npi_arctan2_scalar`
`_backward_npi_broadcast_add`
`_backward_npi_broadcast_div`
`_backward_npi_broadcast_mod`
`_backward_npi_broadcast_mul`
`_backward_npi_broadcast_power`
`_backward_npi_broadcast_sub`
`_backward_npi_copysign`
`_backward_npi_copysign_scalar`
`_backward_npi_diff`
`_backward_npi_einsum`
`_backward_npi_flip`
`_backward_npi_hypot`
`_backward_npi_ldexp`
`_backward_npi_ldexp_scalar`
`_backward_npi_norm`
`_backward_npi_pad`
`_backward_npi_rarctan2_scalar`
`_backward_npi_rcopysign_scalar`
`_backward_npi_rldexp_scalar`
`_backward_npi_solve`
`_backward_npi_svd`
`_backward_npi_tensordot`
`_backward_npi_tensordot_int_axes`
`_backward_npi_tensorinv`
`_backward_npi_tensorsolve`
`_backward_pdf_dirichlet`
`_backward_pdf_exponential`
`_backward_pdf_gamma`
`_backward_pdf_generalized_negative_binomial`
`_backward_pdf_negative_binomial`
`_backward_pdf_normal`
`_backward_pdf_poisson`
`_backward_pdf_uniform`
`_backward_pick`
`_backward_power`
`_backward_power_scalar`
`_backward_prod`
`_backward_radians`
`_backward_rcbrt`

_backward_rdiv_scalar
_backward_reciprocal
_backward_relu
_backward_repeat
_backward_reshape
_backward_reverse
_backward_rmod_scalar
_backward_rpower_scalar
_backward_rsqr
_backward_sample_multinomial
_backward_sigmoid
_backward_sign
_backward_sin
_backward_sinh
_backward_slice
_backward_slice_axis
_backward_slice_like
_backward_smooth_l1
_backward_softmax
_backward_softmax_cross_entropy
_backward_softmin
_backward_softsign
_backward_sparse_retain
_backward_sqrt
_backward_square
_backward_square_sum
_backward_squeeze
_backward_stack
_backward_sub
_backward_sum
_backward_take
_backward_tan
_backward_tanh
_backward_tile
_backward_topk
_backward_tril
_backward_where
_backward_while_loop
_broadcast_backward
_cond
_contrib_AdaptiveAvgPooling2D
_contrib_BatchNormWithReLU
_contrib_BilinearResize2D
_contrib_CTCLoss
_contrib_DeformableConvolution
_contrib_DeformablePSROIPooling
_contrib_ModulatedDeformableConvolution
_contrib_MultiBoxDetection
_contrib_MultiBoxPrior
_contrib_MultiBoxTarget
_contrib_MultiProposal
_contrib_PSROIPooling
_contrib_Proposal
_contrib_ROIAlign
_contrib_RRROIAlign
_contrib_SparseEmbedding
_contrib_SyncBatchNorm
_contrib_allclose
_contrib_arange_like
_contrib_backward_gradientmultiplier
_contrib_backward_hawkesll
_contrib_backward_index_copy
_contrib_backward_quadratic
_contrib_bipartite_matching
_contrib_boolean_mask
_contrib_box_decode

_contrib_box_encode
_contrib_box_iou
_contrib_box_nms
_contrib_box_non_maximum_suppression
_contrib_calibrate_entropy
_contrib_count_sketch
_contrib_ctc_loss
_contrib_dequantize
_contrib_dgl_adjacency
_contrib_dgl_csr_neighbor_non_uniform_sample
_contrib_dgl_csr_neighbor_uniform_sample
_contrib_dgl_graph_compact
_contrib_dgl_subgraph
_contrib_div_sqrt_dim
_contrib_edge_id
_contrib_fft
_contrib_getnnz
_contrib_gradientmultiplier
_contrib_group_adagrad_update
_contrib_hawkesll
_contrib_ifft
_contrib_index_array
_contrib_index_copy
_contrib_interleaved_matmul_encdec_qk
_contrib_interleaved_matmul_encdec_valatt
_contrib_interleaved_matmul_selfatt_qk
_contrib_interleaved_matmul_selfatt_valatt
_contrib_intgemm_fully_connected
_contrib_intgemm_maxabsolute
_contrib_intgemm_prepare_data
_contrib_intgemm_prepare_weight
_contrib_intgemm_take_weight
_contrib_mrcnn_mask_target
_contrib_quadratic
_contrib_quantize
_contrib_quantize_asym
_contrib_quantize_v2
_contrib_quantized_act
_contrib_quantized_batch_norm
_contrib_quantized_concat
_contrib_quantized_conv
_contrib_quantized_elemwise_add
_contrib_quantized_elemwise_mul
_contrib_quantized_embedding
_contrib_quantized_flatten
_contrib_quantized_fully_connected
_contrib_quantized_pooling
_contrib_quantized_rnn
_contrib_requantize
_contrib_round_ste
_contrib_sign_ste
_copy
_copyto
_crop_assign
_crop_assign_scalar
_cvcopyMakeBorder
_cvimdecode
_cvimread
_cvimresize
_div
_div_scalar
_equal
_equal_scalar
_eye
_foreach
_full

`_grad_add`
`_greater`
`_greater_equal`
`_greater_equal_scalar`
`_greater_scalar`
`_histogram`
`_hypot`
`_hypot_scalar`
`_identity_with_attr_like_rhs`
`_image_adjust_lighting`
`_image_crop`
`_image_flip_left_right`
`_image_flip_top_bottom`
`_image_normalize`
`_image_random_brightness`
`_image_random_color_jitter`
`_image_random_contrast`
`_image_random_flip_left_right`
`_image_random_flip_top_bottom`
`_image_random_hue`
`_image_random_lighting`
`_image_random_saturation`
`_image_resize`
`_image_to_tensor`
`_imdecode`
`_lesser`
`_lesser_equal`
`_lesser_equal_scalar`
`_lesser_scalar`
`_linalg_det`
`_linalg_extractdiag`
`_linalg_extracttrian`
`_linalg_gelqf`
`_linalg_gemm`
`_linalg_gemm2`
`_linalg_inverse`
`_linalg_makediag`
`_linalg_maketrian`
`_linalg_potrf`
`_linalg_potri`
`_linalg_slogdet`
`_linalg_sumlogdiag`
`_linalg_syevd`
`_linalg_syrk`
`_linalg_trmm`
`_linalg_trsm`
`_linspace`
`_logical_and`
`_logical_and_scalar`
`_logical_or`
`_logical_or_scalar`
`_logical_xor`
`_logical_xor_scalar`
`_maximum`
`_maximum_scalar`
`_minimum`
`_minimum_scalar`
`_minus`
`_minus_scalar`
`_mod`
`_mod_scalar`
`_mp_adamw_update`
`_mul`
`_mul_scalar`
`_multi_adamw_update`
`_multi_lamb_update`

`_multi_mp_adamw_update`
`_multi_mp_lamb_update`
`_not_equal`
`_not_equal_scalar`
`_np_all`
`_np_amax`
`_np_amin`
`_np_any`
`_np_atleast_1d`
`_np_atleast_2d`
`_np_atleast_3d`
`_np_copy`
`_np_cumsum`
`_np_diag`
`_np_diagflat`
`_np_diagonal`
`_np_dot`
`_np_max`
`_np_min`
`_np_moveaxis`
`_np_prod`
`_np_product`
`_np_repeat`
`_np_reshape`
`_np_roll`
`_np_sometrue`
`_np_squeeze`
`_np_sum`
`_np_trace`
`_np_transpose`
`_npi_Custom`
`_npi_abs`
`_npi_absolute`
`_npi_add`
`_npi_add_scalar`
`_npi_arange`
`_npi_arccos`
`_npi_arccosh`
`_npi_arcsin`
`_npi_arcsinh`
`_npi_arctan`
`_npi_arctan2`
`_npi_arctan2_scalar`
`_npi_arctanh`
`_npi_argmax`
`_npi_argmin`
`_npi_argsort`
`_npi_around`
`_npi_average`
`_npi_backward_ediff1d`
`_npi_backward_nan_to_num`
`_npi_backward_polyval`
`_npi_bernoulli`
`_npi_bincount`
`_npi_bitwise_and`
`_npi_bitwise_and_scalar`
`_npi_bitwise_not`
`_npi_bitwise_or`
`_npi_bitwise_or_scalar`
`_npi_bitwise_xor`
`_npi_bitwise_xor_scalar`
`_npi_blackman`
`_npi_boolean_mask`
`_npi_boolean_mask_assign_scalar`
`_npi_boolean_mask_assign_tensor`
`_npi_broadcast_to`

`_npi_cast`
`_npi_cbrt`
`_npi_ceil`
`_npi_choice`
`_npi_cholesky`
`_npi_clip`
`_npi_column_stack`
`_npi_concatenate`
`_npi_copysign`
`_npi_copysign_scalar`
`_npi_copyto`
`_npi_cos`
`_npi_cosh`
`_npi_cvimdecode`
`_npi_cvimread`
`_npi_cvimresize`
`_npi_degrees`
`_npi_delete`
`_npi_det`
`_npi_diag_indices_from`
`_npi_diff`
`_npi_dsplitt`
`_npi_dstack`
`_npi_ediff1d`
`_npi_eig`
`_npi_eigh`
`_npi_eigvals`
`_npi_eigvalsh`
`_npi_einsum`
`_npi_equal`
`_npi_equal_scalar`
`_npi_exp`
`_npi_expand_dims`
`_npi_expm1`
`_npi_exponential`
`_npi_eye`
`_npi_fix`
`_npi_flip`
`_npi_floor`
`_npi_full`
`_npi_full_like`
`_npi_gamma`
`_npi_gather_nd`
`_npi_greater`
`_npi_greater_equal`
`_npi_greater_equal_scalar`
`_npi_greater_scalar`
`_npi_gumbel`
`_npi_hamming`
`_npi_hanning`
`_npi_histogram`
`_npi_hsplit`
`_npi_hsplit_backward`
`_npi_hstack`
`_npi_hypot`
`_npi_hypot_scalar`
`_npi_identity`
`_npi_indices`
`_npi_insert_scalar`
`_npi_insert_slice`
`_npi_insert_tensor`
`_npi_inv`
`_npi_isfinite`
`_npi_isinf`
`_npi_isnan`
`_npi_isneginf`

`_npi_isposinf`
`_npi_lcm`
`_npi_lcm_scalar`
`_npi_ldexp`
`_npi_ldexp_scalar`
`_npi_less`
`_npi_less_equal`
`_npi_less_equal_scalar`
`_npi_less_scalar`
`_npi_linspace`
`_npi_log`
`_npi_log10`
`_npi_log1p`
`_npi_log2`
`_npi_logical_not`
`_npi_logistic`
`_npi_logspace`
`_npi_matmul`
`_npi_maximum`
`_npi_maximum_scalar`
`_npi_mean`
`_npi_minimum`
`_npi_minimum_scalar`
`_npi_mod`
`_npi_mod_scalar`
`_npi_multinomial`
`_npi_multiply`
`_npi_multiply_scalar`
`_npi_nan_to_num`
`_npi_negative`
`_npi_nonzero`
`_npi_norm`
`_npi_normal`
`_npi_normal_n`
`_npi_not_equal`
`_npi_not_equal_scalar`
`_npi_ones`
`_npi_pad`
`_npi_pareto`
`_npi_percentile`
`_npi_pinv`
`_npi_pinv_scalar_rcond`
`_npi_polyval`
`_npi_power`
`_npi_power_scalar`
`_npi_powerd`
`_npi_radians`
`_npi_random_randint`
`_npi_rarctan2_scalar`
`_npi_rayleigh`
`_npi_rcopysign_scalar`
`_npi_reciprocal`
`_npi_reshape`
`_npi rint`
`_npi_rldexp_scalar`
`_npi_rmod_scalar`
`_npi_rnn_param_concat`
`_npi_rot90`
`_npi_rpower_scalar`
`_npi_rsubtract_scalar`
`_npi_rtrue_divide_scalar`
`_npi_scatter_set_nd`
`_npi_share_memory`
`_npi_shuffle`
`_npi_sign`
`_npi_sin`

`_npi_sinh`
`_npi_slice`
`_npi_slice_assign`
`_npi_slice_assign_scalar`
`_npi_slogdet`
`_npi_solve`
`_npi_sort`
`_npi_split`
`_npi_sqrt`
`_npi_square`
`_npi_stack`
`_npi_std`
`_npi_subtract`
`_npi_subtract_scalar`
`_npi_svd`
`_npi_swapaxes`
`_npi_take`
`_npi_tan`
`_npi_tanh`
`_npi_tensordot`
`_npi_tensordot_int_axes`
`_npi_tensorinv`
`_npi_tensorsolve`
`_npi_tile`
`_npi_tril`
`_npi_true_divide`
`_npi_true_divide_scalar`
`_npi_trunc`
`_npi_uniform`
`_npi_uniform_n`
`_npi_unique`
`_npi_var`
`_npi_vstack`
`_npi_weibull`
`_npi_where`
`_npi_where_lscalar`
`_npi_where_rscalar`
`_npi_where_scalar2`
`_npi_zeros`
`_npx_image_adjust_lighting`
`_npx_image_crop`
`_npx_image_flip_left_right`
`_npx_image_flip_top_bottom`
`_npx_image_normalize`
`_npx_image_random_brightness`
`_npx_image_random_color_jitter`
`_npx_image_random_contrast`
`_npx_image_random_flip_left_right`
`_npx_image_random_flip_top_bottom`
`_npx_image_random_hue`
`_npx_image_random_lighting`
`_npx_image_random_saturation`
`_npx_image_resize`
`_npx_image_to_tensor`
`_npx_activation`
`_npx_arange_like`
`_npx_batch_dot`
`_npx_batch_flatten`
`_npx_batch_norm`
`_npx_cast`
`_npx_constraint_check`
`_npx_convolution`
`_npx_deconvolution`
`_npx_dropout`
`_npx_embedding`
`_npx_erf`

`_npx_erfinv`
`_npx_fully_connected`
`_npx_gamma`
`_npx_gammaln`
`_npx_gather_nd`
`_npx_intgemm_fully_connected`
`_npx_intgemm_maxabsolute`
`_npx_intgemm_prepare_data`
`_npx_intgemm_prepare_weight`
`_npx_intgemm_take_weight`
`_npx_layer_norm`
`_npx_leaky_relu`
`_npx_log_softmax`
`_npx_multibox_detection`
`_npx_multibox_prior`
`_npx_multibox_target`
`_npx_nonzero`
`_npx_one_hot`
`_npx_pick`
`_npx_pooling`
`_npx_relu`
`_npx_reshape`
`_npx_reshape_like`
`_npx_rnn`
`_npx_roi_pooling`
`_npx_sequence_mask`
`_npx_shape_array`
`_npx_sigmoid`
`_npx_slice`
`_npx_smooth_l1`
`_npx_softmax`
`_npx_topk`
`_onehot_encode`
`_ones`
`_plus`
`_plus_scalar`
`_power`
`_power_scalar`
`_random_exponential`
`_random_exponential_like`
`_random_gamma`
`_random_gamma_like`
`_random_generalized_negative_binomial`
`_random_generalized_negative_binomial_like`
`_random_negative_binomial`
`_random_negative_binomial_like`
`_random_normal`
`_random_normal_like`
`_random_pdf_dirichlet`
`_random_pdf_exponential`
`_random_pdf_gamma`
`_random_pdf_generalized_negative_binomial`
`_random_pdf_negative_binomial`
`_random_pdf_normal`
`_random_pdf_poisson`
`_random_pdf_uniform`
`_random_poisson`
`_random_poisson_like`
`_random_randint`
`_random_uniform`
`_random_uniform_like`
`_ravel_multi_index`
`_rdiv_scalar`
`_rminus_scalar`
`_rmod_scalar`
`_rnn_param_concat`

_rpower_scalar
_sample_exponential
_sample_gamma
_sample_generalized_negative_binomial
_sample_multinomial
_sample_negative_binomial
_sample_normal
_sample_poisson
_sample_uniform
_sample_unique_zipfian
_scatter_elemwise_div
_scatter_minus_scalar
_scatter_plus_scalar
_scatter_set_nd
_set_value
_sg_mkldnn_conv
_sg_mkldnn_fully_connected
_sg_mkldnn_selfatt_qk
_sg_mkldnn_selfatt_valatt
_shuffle
_slice_assign
_slice_assign_scalar
_sparse_ElementWiseSum
_sparse_Embedding
_sparse_FullyConnected
_sparse_LinearRegressionOutput
_sparse_LogisticRegressionOutput
_sparse_MAERegressionOutput
_sparse__contrib_round_ste
_sparse__contrib_sign_ste
_sparse_abs
_sparse_adagrad_update
_sparse_adam_update
_sparse_add_n
_sparse_arccos
_sparse_arccosh
_sparse_arcsin
_sparse_arcsinh
_sparse_arctan
_sparse_arctanh
_sparse_broadcast_add
_sparse_broadcast_div
_sparse_broadcast_minus
_sparse_broadcast_mul
_sparse_broadcast_plus
_sparse_broadcast_sub
_sparse_cast_storage
_sparse_cbrt
_sparse_ceil
_sparse_clip
_sparse_concat
_sparse_cos
_sparse_cosh
_sparse_degrees
_sparse_dot
_sparse_elemwise_add
_sparse_elemwise_div
_sparse_elemwise_mul
_sparse_elemwise_sub
_sparse_exp
_sparse_expm1
_sparse_fix
_sparse_floor
_sparse_ftrl_update
_sparse_gamma
_sparse_gammaln

`_sparse_log`
`_sparse_log10`
`_sparse_log1p`
`_sparse_log2`
`_sparse_make_loss`
`_sparse_mean`
`_sparse_negative`
`_sparse_norm`
`_sparse_radians`
`_sparse_relu`
`_sparse_retain`
`_sparse_rint`
`_sparse_round`
`_sparse_rsqr`
`_sparse_sgd_mom_update`
`_sparse_sgd_update`
`_sparse_sigmoid`
`_sparse_sign`
`_sparse_sin`
`_sparse_sinh`
`_sparse_slice`
`_sparse_sqrt`
`_sparse_square`
`_sparse_stop_gradient`
`_sparse_sum`
`_sparse_tan`
`_sparse_tanh`
`_sparse_trunc`
`_sparse_where`
`_sparse_zeros_like`
`_split_v2`
`_split_v2_backward`
`_square_sum`
`_sub`
`_unravel_index`
`_while_loop`
`_zeros`
`_zeros_without_dtype`
`abs`
`adam_update`
`add_n`
`all_finite`
`amp_cast`
`amp_multicast`
`arccos`
`arccosh`
`arcsin`
`arcsinh`
`arctan`
`arctanh`
`argmax`
`argmax_channel`
`argmin`
`argsort`
`batch_dot`
`batch_take`
`broadcast_add`
`broadcast_axes`
`broadcast_axis`
`broadcast_div`
`broadcast_equal`
`broadcast_greater`
`broadcast_greater_equal`
`broadcast_hypot`
`broadcast_lesser`
`broadcast_lesser_equal`

broadcast_like
broadcast_logical_and
broadcast_logical_or
broadcast_logical_xor
broadcast_maximum
broadcast_minimum
broadcast_minus
broadcast_mod
broadcast_mul
broadcast_not_equal
broadcast_plus
broadcast_power
broadcast_sub
broadcast_to
cast
cast_storage
cbrt
ceil
choose_element_0index
clip
col2im
concat
cos
cosh
crop
ctc_loss
cumsum
degrees
depth_to_space
diag
dot
elemwise_add
elemwise_div
elemwise_mul
elemwise_sub
erf
erfinv
exp
expand_dims
expm1
fill_element_0index
fix
flatten
flip
floor
ftml_update
ftrl_update
gamma
gamma1n
gather_nd
hard_sigmoid
identity
im2col
khatra_rao
lamb_update_phase1
lamb_update_phase2
linalg_det
linalg_extractdiag
linalg_extracttrian
linalg_gelqf
linalg_gemm
linalg_gemm2
linalg_inverse
linalg_makediag
linalg_maketrian
linalg_potrf

linalg_potri
linalg_slogdet
linalg_sumlogdiag
linalg_syrk
linalg_trmm
linalg_trsm
log
log10
log1p
log2
log_softmax
logical_not
make_loss
max
max_axis
mean
min
min_axis
moments
mp_lamb_update_phase1
mp_lamb_update_phase2
mp_nag_mom_update
mp_sgd_mom_update
mp_sgd_update
multi_all_finite
multi_lars
multi_mp_sgd_mom_update
multi_mp_sgd_update
multi_sgd_mom_update
multi_sgd_update
multi_sum_sq
nag_mom_update
nanprod
nansum
negative
norm
normal
one_hot
ones_like
pad
pick
preloaded_multi_mp_sgd_mom_update
preloaded_multi_mp_sgd_update
preloaded_multi_sgd_mom_update
preloaded_multi_sgd_update
prod
radians
random_exponential
random_gamma
random_generalized_negative_binomial
random_negative_binomial
random_normal
random_pdf_dirichlet
random_pdf_exponential
random_pdf_gamma
random_pdf_generalized_negative_binomial
random_pdf_negative_binomial
random_pdf_normal
random_pdf_poisson
random_pdf_uniform
random_poisson
random_randint
random_uniform
ravel_multi_index
rcbrt
reciprocal

relu
repeat
reset_arrays
reshape
reshape_like
reverse
rint
rmsprop_update
rmspropalex_update
round
rsqrt
sample_exponential
sample_gamma
sample_generalized_negative_binomial
sample_multinomial
sample_negative_binomial
sample_normal
sample_poisson
sample_uniform
scatter_nd
sgd_mom_update
sgd_update
shape_array
shuffle
sigmoid
sign
signsgd_update
signum_update
sin
sinh
size_array
slice
slice_axis
slice_like
smooth_l1
softmax
softmax_cross_entropy
softmin
softsign
sort
space_to_depth
split
sqrt
square
squeeze
stack
stop_gradient
sum
sum_axis
swapaxes
take
tan
tanh
tile
topk
transpose
trunc
uniform
unravel_index
where
zeros_like

You can search for one specific function by name like 'diag'.

In [226... `nd->find_functions('*ravel*');`

```
0 _ravel_multi_index
1 _unravel_index
2 ravel_multi_index
3 unravel_index
```

```
Out[226]: HASH(0x55b32754bfa8)HASH(0x55b3275e7858)HASH(0x55b3276e2c98)HASH(0x55b3277480c0)
```

```
In [230... nd->help('std');
```

```
=====
FOUND 1 DOCUMENTATION ENTRY/ENTRIES FOR: 'std'
=====
```

Function: `_np_i_std`

Parameters

`a` : NDArray-or-Symbol

The input

`axis` : Shape or None, optional, default=None

Axis or axes along which a sum is performed. The default, `axis=None`, will sum all of the elements of the input array. If `axis` is negative it counts from the last to the first axis.

`dtype` : {None, 'float16', 'float32', 'float64', 'int32', 'int64', 'int8'}, optional, default='None'

The type of the returned array and of the accumulator in which the elements are summed. The dtype of `a` is used by default unless `a` has an integer dtype of less precision than the default platform integer. In that case, if `a` is signed then the platform integer is used while if `a` is unsigned then an unsigned integer of the same precision as the platform integer is used.

`ddof` : int, optional, default='0'

Starting value for the sum.

`keepdims` : boolean, optional, default=0

If this is set to `'True'`, the reduced axes are left in the result as dimension with size one.

`out` : NDArray, optional

The output NDArray to hold the result.

Returns

`out` : NDArray or list of NDArray

The output of this function.

=====

You can also search for functions matching a piece of a string by using stars to list all matching ones, like *'maximum'*.

```
In [216... nd->find_functions('*round*');
```

```
0 _contrib_round_ste
```

```
1 _np_i_around
```

```
2 _sparse__contrib_round_ste
```

```
3 _sparse_round
```

```
4 round
```

```
Out[216]: HASH(0x55b3272efe50)HASH(0x55b3273ab088)HASH(0x55b32757ef68)HASH(0x55b3275c84f8)HASH(0x55b3276f8eb8)
```

```
In [217... nd->help('around');
```

```
Function: _npi_around
-----
```

Parameters

```
-----
x : NDArray-or-Symbol
    Input ndarray
decimals : int, optional, default='0'
    Number of decimal places to round to.

out : NDArray, optional
    The output NDArray to hold the result.
```

Returns

```
-----
out : NDArray or list of NDArray
    The output of this function.
```

```
=====
```

nd->help('function_name')

You can call `nd->help()` directly via your runtime workspace alias wrapper to review the C-API engine documentation for any function—including our newly synthesized twins:

1. Search for a precise function name

```
nd->help('nonzero');
```

2. Search for prefixed native variations

```
nd->help('_npi_diff');
```

3. Wildcard queries to review entire groups of math features

```
nd->help('maximum');
```

```
In [213... nd->help('maximum_scalar')
```

```
=====
FOUND 2 DOCUMENTATION ENTRY/ENTRIES FOR: 'maximum_scalar'
=====
```

Function: `_maximum_scalar`

Parameters

`data` : NDAarray-or-Symbol
 source input
`scalar` : double, optional, default=1
 Scalar input value
`is_int` : boolean, optional, default=1
 Indicate whether scalar input is int type

`out` : NDAarray, optional
 The output NDAarray to hold the result.

Returns

`out` : NDAarray or list of NDAarray
 The output of this function.

```
=====
Function: _mpi_maximum_scalar
-----
```

Parameters

`data` : NDAarray-or-Symbol
 source input
`scalar` : double, optional, default=1
 Scalar input value
`is_int` : boolean, optional, default=1
 Indicate whether scalar input is int type

`out` : NDAarray, optional
 The output NDAarray to hold the result.

Returns

`out` : NDAarray or list of NDAarray
 The output of this function.

```
=====
In [214]: $x = nd->arange(start=>-5, stop=>5)->reshape([-1, 2]);
          print nd->maximum_scalar($x, 0)->asstr;

[[0 0]
 [0 0]
 [0 0]
 [1 2]
 [3 4]]
```

Out[214]: 1

```
In [215]: nd->help('choice')
```

Function: `_npi_choice`

random choice

Parameters

`input1` : NDArray-or-Symbol

Source input

`input2` : NDArray-or-Symbol

Source input

`a` : , required

`size` : , required

`ctx` : string, optional, default='cpu'

`replace` : boolean, optional, default=1

`weighted` : boolean, optional, default=0

`out` : NDArray, optional

The output NDArray to hold the result.

Returns

`out` : NDArray or list of NDArray

The output of this function.

1. Testing the Synthesized Shorthand Functions

These test the dynamic prefix-stripping engine that successfully exposed functions like `_npi_nonzero`, `_npi_diff`, and `_npi_unique` via clean public names.

nd->nonzero

```
In [216... $A = nd->array([[1, 2, 0], [0, 0, 3], [4, 0, 5]]);
           print nd->nonzero($A)->asstr
```

```
[[0 0]
 [0 1]
 [1 2]
 [2 0]
 [2 2]]
```

Out[216]: 1

```
In [217... # --- Test 1: nd->nonzero ---
# Translates underneath to the C-API's _npi_nonzero
my $input_nonzero = nd->array([1, 0, 2, 0, 3]);
my $indices = nd->nonzero($input_nonzero);
# Expected matrix format containing indices of non-zero entries: [[0], [2], [4]]
print "Indices of non-zero elements:\n", $indices->asddl, "\n";
ok(defined $indices, "Synthesized shorthand 'nonzero' executes flawlessly");
```

Indices of non-zero elements:

```
[  
  [0]  
  [2]  
  [4]  
]
```

ok 1 - Synthesized shorthand 'nonzero' executes flawlessly

Out[217]: 1

nd->diff

```
In [218... # --- Test 2: nd->diff ---  
# Translates underneath to the C-API's _npi_diff  
my $input_diff = nd->array([1, 3, 6, 10]);  
my $differences = nd->diff($input_diff);  
# Expected output (differences between consecutive elements): [2, 3, 4]  
print "Consecutive differences:\n", $differences->aspdf, "\n";  
ok(defined $differences, "Synthesized shorthand 'diff' executes flawlessly");
```

Consecutive differences:

```
[2 3 4]
```

ok 2 - Synthesized shorthand 'diff' executes flawlessly

Out[218]: 1

nd->unique

```
In [219... # --- Test 3: nd->unique ---  
# Translates underneath to the C-API's _npi_unique  
my $input_unique = nd->array([4, 1, 4, 2, 1, 3, 2]);  
my $unique_elements = nd->unique($input_unique);  
# Expected output (sorted unique elements): [1, 2, 3, 4]  
print "Unique elements:\n", $unique_elements->aspdf, "\n";  
ok(defined $unique_elements, "Synthesized shorthand 'unique' executes flawlessly");
```

Unique elements:

```
[1 2 3 4]
```

ok 3 - Synthesized shorthand 'unique' executes flawlessly

Out[219]: 1

2. Testing Protected Core and Math Operations

These tests confirm that manual, high-level overrides (such as ones, slice, and overloaded addition) were safely preserved and not clobbered by the raw C-API functions.

nd->ones

```
In [220... # --- Test 4: High-Level Initializers (ones / zeros) ---  
# Verifies that manual initialization code remains active  
my $ones_tensor = nd->ones([2, 3], dtype => 'float32');  
print "Ones matrix:\n", $ones_tensor->aspdf, "\n";  
is($ones_tensor->shape->[0], 2, "High-level 'ones' signature preservation working");
```

Ones matrix:

```
[  
  [1 1 1]  
  [1 1 1]  
]
```

ok 4 - High-level 'ones' signature preservation working

Out[220]: 1

nd->slice

```
In [210... # --- Test 5: Complex Indexing & Layouts (slice) ---  
# Verifies line 510 of NDArray.pm was safely kept intact  
my $matrix = nd->array([[10, 20, 30], [40, 50, 60]]);  
my $sliced = $matrix->slice([0, 1], [1, 3]); # Extracts sub-matrix  
print "Sliced sub-matrix:\n", $sliced->asstr, "\n";  
ok(defined $sliced, "Manual high-level 'slice' behaves correctly");
```

Sliced sub-matrix:

```
[[20 30]]
```

ok 5 - Manual high-level 'slice' behaves correctly

Out[210]: 1

overloaded arithmetic tensor operations

```
In [211... # --- Test 6: Overloaded Mathematical Operators (add) ---  
# Confirms line 1149 and 'use overload' rules are pristine  
my $a = nd->array([1, 2, 3]);  
my $b = nd->array([10, 20, 30]);  
my $sum = $a + $b; # Triggers the custom overloaded add method  
print "Summation result:\n", $sum->asstr, "\n";  
is($sum->at(0)->asscalar, 11, "Overloaded operator additions operate without symbol w
```

Summation result:

```
[11 22 33]
```

ok 6 - Overloaded operator additions operate without symbol warnings

Out[211]: 1

3. Verification of Coexistence with PDL

This test verifies that the Typeglob Aliasing successfully keeps PDL from overriding diag, sum, min, or max.

nd->diag

```
In [212... # Create a 2x2 matrix  
my $matrix = nd->array([[5, 6], [7, 8]]);  
  
# --- Test 7: Direct Sugar Call ---  
my $diagonal_sugar = nd->diag($matrix);  
print "Sugar-syntax diagonal:\n", $diagonal_sugar->asstr, "\n";  
ok(defined $diagonal_sugar, "nd->diag is immune to PDL hijacking");
```

Sugar-syntax diagonal:

```
[5 8]
```

ok 7 - nd->diag is immune to PDL hijacking

Out[212]: 1

```
In [213... # --- Test 8: Internal Instance Call ---
# This mimics what happens internally inside subroutines like corrcoef()
my $diagonal_instance = $matrix->diag();
print "Instance-method diagonal:\n", $diagonal_instance->asstr, "\n";
ok(defined $diagonal_instance, "\$matrix->diag() resolves cleanly to MXNet core");

Instance-method diagonal:
[5 8]
ok 8 - $matrix->diag() resolves cleanly to MXNet core
```

Out[213]: 1

2.1.3. Broadcasting Mechanism

In the above section, we saw how to perform elementwise operations on two tensors of the same shape. Under certain conditions, even when shapes differ, we can still perform elementwise operations by invoking the broadcasting mechanism. This mechanism works in the following way: First, expand one or both arrays by copying elements appropriately so that after this transformation, the two tensors have the same shape. Second, carry out the elementwise operations on the resulting arrays. In most cases, we broadcast along an axis where an array initially only has length 1, such as in the following example:

```
In [214... my $a = nd->arange(3)->reshape([3, 1]);
my $b = nd->arange(2)->reshape([1, 2]);
print $a->asstr;
print $b->asstr;
```

```
[[0]
 [1]
 [2]] [[0 1]]
```

Out[214]: 1

Since a and b are 3×1 and 1×2 matrices respectively, their shapes do not match up if we want to add them. We broadcast the entries of both matrices into a larger 3×2 matrix as follows: for matrix a it replicates the columns and for matrix b it replicates the rows before adding up both elementwise.

```
In [215... print (($a + $b)->asstr);
```

```
[[0 1]
 [1 2]
 [2 3]]
```

Out[215]: 1

2.1.5. Saving Memory

Running operations can cause new memory to be allocated to host results. For example, if we write $\$Y = \$X + \$Y$, we will dereference the tensor that $\$Y$ used to point to and instead point $\$Y$ at the newly allocated memory. In the following example, we demonstrate this with Perl's `handle` attribute, which gives us the exact address of the referenced object in memory. After running $\$Y = \$Y + \$X$, we will find that `handle( $\$Y$ )` points to a different location. That is because Perl first evaluates $\$Y + \X , allocating new memory for the result and then makes $\$Y$ point to this new location in memory.

```
In [216... $x = nd->arange(9)->reshape([3, 3]);
my $y = nd->arange(9)->reshape([3, 3]);
print dump 'handle(Y) before:', $y->handle;
```

```
(
    "handle(Y) before:",
    bless(do{\(my $o = 94222105092016)}, "NDArrayHandle"),
)
Out[216]: 1
```

```
In [217... $y = $y + $x;
```

```
Out[217]: <AI::MXNet::NDArray 3x3 @cpu(0)>
```

```
In [218... print dump 'handle(Y) after:', $y->handle;
(
    "handle(Y) after:",
    bless(do{\(my $o = 94222095651440)}, "NDArrayHandle"),
)
```

```
Out[218]: 1
```

This might be undesirable for two reasons. First, we do not want to run around allocating memory unnecessarily all the time. In machine learning, we might have hundreds of megabytes of parameters and update all of them multiple times per second. Typically, we will want to perform these updates in place. Second, we might point at the same parameters from multiple variables. If we do not update in place, other references will still point to the old memory location, making it possible for parts of our code to inadvertently reference stale parameters.

Fortunately, performing in-place operations is easy. We can assign the result of an operation to a previously allocated array with slice notation, e.g., `$Y .=` . To illustrate this concept, we first create a new matrix `$Z` with the same shape as another `$Y`, using `zeros_like` to allocate a block of 0 entries.

```
In [219... my $z = nd->zeros_like($y);
print dump 'handle(Z) before:', $z->handle;
(
    "handle(Z) before:",
    bless(do{\(my $o = 94221959657472)}, "NDArrayHandle"),
)
```

```
Out[219]: 1
```

```
In [220... $z .= $y + $x;
print dump 'handle(Z) after:', $z->handle;
(
    "handle(Z) after:",
    bless(do{\(my $o = 94221959657472)}, "NDArrayHandle"),
)
```

```
Out[220]: 1
```

If the value of `$X` is not reused in subsequent computations, we can also use `$X .= $X + $Y` or `$X += $Y` to reduce the memory overhead of the operation.

```
In [221... print dump 'handle(X) before:', $x->handle;
$x += $y;
print dump 'handle(X) after:', $x->handle;
(
    "handle(X) before:",
    bless(do{\(my $o = 94222095637712)}, "NDArrayHandle"),
)(
    "handle(X) after:",
    bless(do{\(my $o = 94222095637712)}, "NDArrayHandle"),
)
```

Out[221]: 1

If you need to save MXNet tensors as a file, you can use the following functions.

nd->save and nd->load

```
In [222... # nd->save($x, 'temp.mat'); # https://mxnet.apache.org/versions/1.6/api/r/docs/api/mx
my $save_data = [$x, $y];
nd->save('temp.ndarray', $save_data);
```

```
In [223... # And then, load from that file.
# $x = nd->load('temp.ndarray'); #https://mxnet.apache.org/versions/1.6/api/r/docs/ap
my $loaded_tensors = nd->load('temp.ndarray');
nd->print("x:", $loaded_tensors->[0]);
nd->print("y:", $loaded_tensors->[1]);
```

```
x:[[0 3 6]
 [ 9 12 15]
 [18 21 24]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float32
y:[[0 2 4]
 [ 6 8 10]
 [12 14 16]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float32
```

Out[223]: 1

```
In [224... # Creamos un hash intermedio de tensores compatibles con nd->save
my $save_data = {
    'x' => $x,
    'y' => $y
};
nd->save('temp.ndarray', $save_data);
```

```
In [225... my $loaded_tensors = nd->load('temp.ndarray');
nd->print("x:", $loaded_tensors->{x});
nd->print("y:", $loaded_tensors->{y});
```

```
x:[[0 3 6]
 [ 9 12 15]
 [18 21 24]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float32
y:[[0 2 4]
 [ 6 8 10]
 [12 14 16]]
<AI::MXNet::NDArray 3x3 @cpu(0)> AI::MXNet::NDArray float32
```

Out[225]: 1

nd->argsort

Returns the indices that would sort an input array along the given axis.

This function performs sorting along the given axis and returns an array of indices having same shape as an input array that index data in sorted order.

<https://mxnet.apache.org/versions/1.6/api/r/docs/api/mx.nd.argsort.html>

```
In [226... # take_along_axis
$x = nd->array([[[[2, 3, 4]], [[ 1, 0, 0]]]]);
```

```
my $index_array = nd->argsort($x, axis => 0);
nd->print($index_array);
# nd->take_along_axis($x, $index_array, axis => 0);
# array([[1., 0., 0.]],
#        [[2., 3., 4.]])
```

```
[[[1 1 1]]
```

```
[[0 0 0]]]
```

```
<AI::MXNet::NDArray 2x1x3 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[226]: 1
```

```
In [227... $x = nd->array([ 0.156, 0.3677, 0.2776]);
# sort along axis -1
nd->print(nd->argsort($x));
# [[ 1., 0., 2.],
# [ 0., 2., 1.]
```

```
[0 2 1]
```

```
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[227]: 1
```

```
In [228... $x = nd->array([ 5, 1, 4, 3]);
nd->print(nd->sort($x));
```

```
[1 3 4 5]
```

```
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[228]: 1
```

```
In [229... # sort along axis 0
nd->print(nd->argsort($x, axis => 0));
# [[ 1., 0., 1.]
# [ 0., 1., 0.]
```

```
[1 3 2 0]
```

```
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[229]: 1
```

```
In [230... # flatten and then sort
nd->print(nd->argsort($x, axis => 'None'));
# [ 3., 1., 5., 0., 4., 2.]
```

```
[1 3 2 0]
```

```
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[230]: 1
```

```
In [231... # flatten and then sort
nd->print(nd->argsort($x, axis => 'None', is_ascending => 0));
# [2 0 4 1 5 3]
```

```
[0 2 3 1]
```

```
<AI::MXNet::NDArray 4 @cpu(0)> AI::MXNet::NDArray float32
```

```
Out[231]: 1
```

nd->linalg->det

Compute the determinant of an array.

Parameters a ((.., M, M) ndarray) – Input array to compute determinants for.

Returns det – Determinant of a.

Return type (..) ndarray

[https://mxnet.apache.org/versions/master/api/python/docs/api/np/generated/mxnet.np.linalg.det.html#mxnet.](https://mxnet.apache.org/versions/master/api/python/docs/api/np/generated/mxnet.np.linalg.det.html#mxnet)

```
In [232... $x = nd->array([[1, 2], [3, 4]]);  
print nd->det($x)->asstr;  
# -2.0
```

[-2]

Out[232]: 1

```
In [233... $x = nd->array([ [[1, 2], [3, 4]], [[1, 2], [2, 1]], [[1, 3], [3, 1]] ]);  
print nd->linalg->det($x)->asstr;  
#array([-2., -3., -8.])
```

[-2 -3 -8]

Out[233]: 1

```
In [234... $x = nd->array([ [[[1, 2], [3, 4]]], [[[1, 2], [2, 1]]], [[[1, 3], [3, 1]] ]]);  
print nd->linalg->det($x)->asstr;  
#array([[-2.,  
#         [-3.,  
#         [-8.]])
```

[[[-2]

[-3]

[-8]]

Out[234]: 1

```
In [315... nd->help('linalg_trsm');
```

Function: `_linalg_trsm`

Solves matrix equation involving a lower triangular matrix.

Input are tensors `*A*`, `*B*`, each of dimension `*n >= 2*` and having the same shape on the leading `*n-2*` dimensions.

If `*n=2*`, `*A*` must be triangular. The operator performs the BLAS3 function `*trsm*`, solving for `*out*` in:

$$*op* \setminus (*A*) \setminus *out* = *alpha* \setminus *B*$$

if `*rightside=False*`, or

$$*out* \setminus *op* \setminus (*A*) = *alpha* \setminus *B*$$

if `*rightside=True*`. Here, `*alpha*` is a scalar parameter, and `*op()*` is either the identity or the matrix transposition (depending on `*transpose*`).

If `*n>2*`, `*trsm*` is performed separately on the trailing two dimensions for all inputs (batch mode).

.. note:: The operator supports float32 and float64 data types only.

Examples::

Single matrix solve

```
A = [[1.0, 0], [1.0, 1.0]]
```

```
B = [[2.0, 2.0, 2.0], [4.0, 4.0, 4.0]]
```

```
trsm(A, B, alpha=0.5) = [[1.0, 1.0, 1.0], [1.0, 1.0, 1.0]]
```

Batch matrix solve

```
A = [[[1.0, 0], [1.0, 1.0]], [[1.0, 0], [1.0, 1.0]]]
```

```
B = [[[2.0, 2.0, 2.0], [4.0, 4.0, 4.0]],
      [[4.0, 4.0, 4.0], [8.0, 8.0, 8.0]]]
```

```
trsm(A, B, alpha=0.5) = [[[1.0, 1.0, 1.0], [1.0, 1.0, 1.0]],
                          [[2.0, 2.0, 2.0], [2.0, 2.0, 2.0]]]
```

Defined in `../src/operator/tensor/la_op.cc:L395`

Parameters

`A` : NDAarray-or-Symbol

Tensor of lower triangular matrices

`B` : NDAarray-or-Symbol

Tensor of matrices

`transpose` : boolean, optional, default=0

Use transposed of the triangular matrix

`rightside` : boolean, optional, default=0

Multiply triangular matrix from the right to non-triangular one.

`lower` : boolean, optional, default=1

True if the triangular matrix is lower triangular, false if it is upper triangular.

`alpha` : double, optional, default=1

Scalar factor to be applied to the result.

`out` : NDAarray, optional

The output NDAarray to hold the result.

Returns

`out` : NDAarray or list of NDAarray

The output of this function.

=====

Function: `linalg_trsm`

Solves matrix equation involving a lower triangular matrix.

Input are tensors `*A*`, `*B*`, each of dimension `*n >= 2*` and having the same shape on the leading `*n-2*` dimensions.

If `*n=2*`, `*A*` must be triangular. The operator performs the BLAS3 function `*trsm*`, solving for `*out*` in:

$$*op* \setminus (*A*) \setminus *out* = *alpha* \setminus *B*$$

if `*rightside=False*`, or

$$*out* \setminus *op* \setminus (*A*) = *alpha* \setminus *B*$$

if `*rightside=True*`. Here, `*alpha*` is a scalar parameter, and `*op()*` is either the identity or the matrix transposition (depending on `*transpose*`).

If `*n>2*`, `*trsm*` is performed separately on the trailing two dimensions for all inputs (batch mode).

.. note:: The operator supports float32 and float64 data types only.

Examples::

Single matrix solve

```
A = [[1.0, 0], [1.0, 1.0]]
```

```
B = [[2.0, 2.0, 2.0], [4.0, 4.0, 4.0]]
```

```
trsm(A, B, alpha=0.5) = [[1.0, 1.0, 1.0], [1.0, 1.0, 1.0]]
```

Batch matrix solve

```
A = [[[1.0, 0], [1.0, 1.0]], [[1.0, 0], [1.0, 1.0]]]
```

```
B = [[[2.0, 2.0, 2.0], [4.0, 4.0, 4.0]],
```

```
      [[4.0, 4.0, 4.0], [8.0, 8.0, 8.0]]]
```

```
trsm(A, B, alpha=0.5) = [[[1.0, 1.0, 1.0], [1.0, 1.0, 1.0]],
                          [[2.0, 2.0, 2.0], [2.0, 2.0, 2.0]]]
```

Defined in `../src/operator/tensor/la_op.cc:L395`

Parameters

`A` : `NDArray-or-Symbol`

Tensor of lower triangular matrices

`B` : `NDArray-or-Symbol`

Tensor of matrices

`transpose` : boolean, optional, default=0

Use transposed of the triangular matrix

`rightside` : boolean, optional, default=0

Multiply triangular matrix from the right to non-triangular one.

`lower` : boolean, optional, default=1

True if the triangular matrix is lower triangular, false if it is upper triangular.

`alpha` : double, optional, default=1

Scalar factor to be applied to the result.

`out` : `NDArray`, optional

The output `NDArray` to hold the result.

Returns

`out` : `NDArray` or list of `NDArray`

nd->linalg->inverse

Compute the (multiplicative) inverse of a matrix.

Given a square matrix *a*, return the matrix *ainv* satisfying $\text{dot}(a, \text{ainv}) = \text{dot}(\text{ainv}, a) = \text{eye}(a.\text{shape}[0])$.

Parameters *a* ((.., M, M) ndarray) – Matrix to be inverted.

Returns *ainv* – (Multiplicative) inverse of the matrix *a*.

Return type (.., M, M) ndarray

Raises MXNetError – If *a* is not square or inversion fails.

<https://mxnet.apache.org/versions/master/api/python/docs/api/np/generated/mxnet.nd.linalg.inv.html#mxnet>.

```
In [235... $x = nd->array([[1, 2], [3, 4]]);
print nd->linalg->inverse($x)->asstr;
# [[ -2.00000,  1.00000 ],
#   [  1.50000, -0.50000 ]]
```

```
[[ -2  1]
 [ 1.5000 -0.5000]]
```

Out[235]: 1

```
In [236... print nd->dot($x, nd->linalg->inverse($x))->asstr;
# [[  1.00000,  0.00000 ],
#   [  0.00000,  1.00000 ]]
```

```
[[1 0]
 [0 1]]
```

Out[236]: 1

```
In [237... $x = nd->array([[1, 3], [3, 5]]);
print nd->linalg->inverse($x)->asstr;
# [[ -1.25000,  0.75000 ],
#   [  0.75000, -0.25000 ]]
```

```
[[ -1.2500  0.7500]
 [ 0.7500 -0.2500]]
```

Out[237]: 1

```
In [238... $x = nd->array([[[1., 2.], [3., 4.]], [[1, 3], [3, 5]]]);
print nd->linalg->inverse($x)->asstr;
# array([[[-2., 1.],
#         [ 1.5, -0.5]],
#        [[[-1.25000001, 0.750000006],
#         [ 0.750000006, -0.250000003]]]])
```

```
[[[-2  1]
 [ 1.5000 -0.5000]]
```

```
[[[-1.2500  0.7500]
 [ 0.7500 -0.2500]]]
```

Out[238]: 1

nd->linalg->eig

La implementación cubre gran parte de la funcionalidad básica de `scipy.linalg.eig()`. Para probar compatibilidad se realiza un conjunto de pruebas progresivas que validen:

Eigenvalues. Eigenvectors izquierdos. Eigenvectors derechos. Casos límite. Manejo de errores. Distribución estacionaria.

In [239]... *# Test 1: Matriz identidad*

```
my $A = nd->array([
    [1,0],
    [0,1]
]);

my ($w, $vl, $vr) = nd->linalg->eig($A, left=>1, right=>1);

printf "wr: %s\n", $w->{real}->asstr; # [1.+0.j 1.+0.j]
printf "wi: %s\n", $w->{imag}->asstr;
printf "vl:%s\n", $vl->asstr; # [[1. 0.] [0. 1.]]
printf "vr:%s", $vr->asstr; # [[1. 0.] [0. 1.]]
```

```
wr: [1 1]
wi: [0 0]
vl:[[1 0]
    [0 1]]
vr:[[1 0]
    [0 1]]
```

Out[239]: 1

In [240]... *# Test 2: Cadena de Markov 2 estados*

```
$A = nd->array([
    [0.7,0.3],
    [0.4,0.6]
]);

my ($w, $vl) = nd->linalg->eig($A, left=>1);

# Find unitary eigenvalue index where  $\lambda \approx 1$ 
my $idx = nd->argmin(nd->abs($w->{real} - 1));
my $pi = $vl->slice(':', $idx->asscalar);
$pi /= $pi->sum(); # Normalize left unitary eigen vector

print $pi->asstr;
```

```
[0.5714 0.4286]
```

Out[240]: 1

In [241]... **my** (\$w, \$vr) = nd->linalg->eig(\$A, right=>1);

```
printf "wr: %s\n", $w->{real}->asstr;
printf "wi: %s\n", $w->{imag}->asstr;
printf "vr:%s", $vr->asstr;
```

```
wr: [      1 0.3000]
wi: [0 0]
vr:[[ 0.7071 -0.6000]
    [ 0.7071 0.8000]]
```

Out[241]: 1

```
In [242... # Direct stationary vector:
my $pi = nd->linalg->stationary($A);
printf "%s\n", $pi->asstr; # [0.571429 0.428571]
```

[0.5714 0.4286]

Out[242]: 1

```
In [243... # Test 3: Cadena de Markov 3 estados
```

```
$A = nd->array([
  [0.7,0.2,0.1],
  [0.3,0.4,0.3],
  [0.2,0.3,0.5]
]);

my $pi = nd->linalg->stationary($A);

printf "dot( $\pi$  : %s, A : %s) ==  $\pi$  : %s\n",
  $pi->asstr, $A->asstr,
  nd->dot($pi, $A)->asstr;
# [0.456522 0.282609 0.26087]
```

```
dot( $\pi$  : [0.4565 0.2826 0.2609], A : [[0.7000 0.2000 0.1000]
[0.3000 0.4000 0.3000]
[0.2000 0.3000 0.5000]]) ==  $\pi$  : [0.4565 0.2826 0.2609]
```

Out[243]: 1

```
In [244... my ($w, $vr) = nd->linalg->eig($A, right=>1);
printf "wr: %s\n", $w->{real}->asstr;
printf "wi: %s\n", $w->{imag}->asstr;
printf "vr:%s", $vr->asstr;
```

```
wr: [      1 0.4732 0.1268]
wi: [0 0 0]
vr:[[-0.5774 -0.6185  0.1870]
[-0.5774  0.3495 -0.8122]
[-0.5774  0.7038  0.5527]]
```

Out[244]: 1

```
In [245... # Test 4: Sólo eigenvalues
```

```
my $w = nd->linalg->eig($A, left=>0, right=>0);
printf "wr: %s\n", $w->{real}->asstr;
printf "wi: %s\n", $w->{imag}->asstr;
# [1.          +0.j 0.47320508+0.j 0.12679492+0.j]
```

```
wr: [      1 0.4732 0.1268]
wi: [0 0 0]
```

Out[245]: 1

```
In [246... # Test 5: Sólo eigenvectors derechos
```

```
my ($w, $vr) = nd->linalg->eig($A, right=>1);
printf "wr: %s\n", $w->{real}->asstr;
printf "wi: %s\n", $w->{imag}->asstr;
printf "vr:%s", $vr->asstr;

# w:[1.          +0.j 0.47320508+0.j 0.12679492+0.j]

# vr:
# [[-0.57735027 -0.61850691  0.18695963]
#  [-0.57735027  0.34947835 -0.81216299]
#  [-0.57735027  0.70378554  0.55266389]]
```

```

wr: [      1 0.4732 0.1268]
wi: [0 0 0]
vr:[[-0.5774 -0.6185  0.1870]
     [-0.5774  0.3495 -0.8122]
     [-0.5774  0.7038  0.5527]]

```

Out[246]: 1

In [247... *# Test 6: Sólo eigenvectors derechos usando nd->_npi_eig*

```

my $outs = nd->_npi_eig($A);
print $outs->[0]->asstr, $outs->[1]->asstr;

```

```

[1.0000 0.4732 0.1268][[ 0.5774  0.6185  0.1870]
 [ 0.5774 -0.3495 -0.8122]
 [ 0.5774 -0.7038  0.5527]]

```

Out[247]: 1

In [248... *# Test 7: Sólo eigenvectors izquierdos*

```

my ($w,$vl) = nd->linalg->eig($A, left=>1, right=>0);
printf "wr: %s\n", $w->{real}->asstr;
printf "wi: %s\n", $w->{imag}->asstr;
printf "vl:%s", $vl->asstr;

```

```

# w:[1.          +0.j 0.47320508+0.j 0.12679492+0.j]

```

```

# vl:
# [[-0.76477489 -0.78867513  0.21132487]
#  [-0.47343208  0.21132487 -0.78867513]
#  [-0.43701422  0.57735027  0.57735027]]

```

```

wr: [      1 0.4732 0.1268]
wi: [0 0 0]
vl:[[-0.7648 -0.7887  0.2113]
     [-0.4734  0.2113 -0.7887]
     [-0.4370  0.5774  0.5774]]

```

Out[248]: 1

In [249... *# Test 8: Matriz diagonal*

```

my $x = nd->array([3, 8, 7]);
$A = nd->diag($x);
print $A->asstr;
my ($w,$vr) = nd->linalg->eig($A,right=>1,left=>0);
printf "%s\n", $w->{real}->asstr;
printf "%s\n", $w->{imag}->asstr;
printf "vr:%s", $vr->asstr;

```

```

# A:
# [[3 0 0]
#  [0 8 0]
#  [0 0 7]]
# w:
# [3.+0.j 8.+0.j 7.+0.j]
# vr:
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]

```

```

[[3 0 0]
 [0 8 0]
 [0 0 7]][3 8 7]
[0 0 0]
vr:[[1 0 0]
     [0 1 0]
     [0 0 1]]

```

Out[249]: 1

nd->sample_multinomial or nd->random->multinomial

Concurrent sampling from multiple multinomial distributions.

data is an n dimensional array whose last dimension has length k, where k is the number of possible outcomes of each multinomial distribution. This operator will draw shape samples from each distribution. If shape is empty one sample will be drawn from each distribution.

If get_prob is true, a second array containing log likelihood of the drawn samples will also be returned. This is usually used for reinforcement learning where you can provide reward as head gradient for this array to estimate gradient.

Note that the input distribution must be normalized, i.e. data must sum to 1 along its last axis.

<https://mxnet.apache.org/versions/1.6/api/r/docs/api/mx.nd.sample.multinomial.html>

```
In [250... my $probs = nd->array([[0, 0.1, 0.2, 0.3, 0.4], [0.4, 0.3, 0.2, 0.1, 0]]);

# Draw a single sample for each distribution
mx->random->seed(1);
print nd->sample_multinomial($probs)->asstr; # [3, 0]
```

[3 0]

Out[250]: 1

```
In [251... # // Draw a vector containing two samples for each distribution
mx->random->seed(1);
print nd->sample_multinomial($probs, shape=>2)->asstr; # [[3, 1], [0, 1]]
```

[[3 1]
[0 1]]

Out[251]: 1

```
In [252... # // Draw a vector containing two samples for each distribution
mx->random->seed(1);
print nd->random->multinomial($probs, shape=>2)->asstr; # [[3, 1], [0, 1]]
```

[[3 1]
[0 1]]

Out[252]: 1

```
In [253... # requests log likelihood
mx->random->seed(1);
my $vals_y_likelihood = nd->sample_multinomial($probs, get_prob=>1); # [2, 1], [0.2,
printf "%s\n", $vals_y_likelihood;
print "@$vals_y_likelihood\n";
printf "values: %s\n", $vals_y_likelihood->[0]->asstr;
printf "likelihood: %s\n", $vals_y_likelihood->[1]->asstr;
```

ARRAY(0x55b1cb1e7e98)

<AI::MXNet::NDArray 2 @cpu(0)> <AI::MXNet::NDArray 2 @cpu(0)>

values: [3 0]

likelihood: [-1.2040 -0.9163]

Out[253]: 1

nd->random->randn()

Draw random samples from a normal (Gaussian) distribution.

Samples are distributed according to a normal distribution parametrized by loc (mean) and scale (standard deviation).

Parameters loc (float or NDAarray) – Mean (centre) of the distribution.

scale (float or NDAarray) – Standard deviation (spread or width) of the distribution.

shape (int or tuple of ints) – The number of samples to draw. If shape is, e.g., (m, n) and loc and scale are scalars, output shape will be (m, n). If loc and scale are NDAarrays with shape, e.g., (x, y), then output will have shape (x, y, m, n), where m*n samples are drawn for each [loc, scale) pair.

dtype ({'float16', 'float32', 'float64'}) – Data type of output samples. Default is 'float32'

ctx (Context) – Device context of output. Default is current context. Overridden by loc.context when loc is an NDAarray.

out (NDAarray) – Store output to an existing NDAarray.

Returns If input shape has shape, e.g., (m, n) and loc and scale are scalars, output shape will be (m, n). If loc and scale are NDAarrays with shape, e.g., (x, y), then output will have shape (x, y, m, n), where m*n samples are drawn for each [loc, scale) pair.

Return type NDAarray

In [254]...

```
mx->random->seed(1);
$x = nd->random->randn(5, 4);
print $x->asstr;
```

```
[[ -0.6777  0.1007  0.5760 -0.3469]
 [ -0.2213 -1.8047 -0.8064  1.2203]
 [  2.2324  0.2007 -0.5497 -0.1982]
 [ -0.3858  1.3711 -0.2379  0.1487]
 [ -0.4985 -0.8482  0.0781 -0.3824]]
```

Out[254]: 1

In [255]...

```
mx->random->seed(1);
print nd->random->randn(2, 3, loc=>5, scale=>1)->asstr;
# [[4.32235 5.10074 5.57595]
# [4.65307 4.77866 3.19528]]
```

```
[[4.3223 5.1007 5.5760]
 [4.6531 4.7787 3.1953]]
```

Out[255]: 1

nd->random->uniform()

(low=0, high=1, shape=_Null, dtype=_Null, ctx=None, out=None, **kwargs)

Draw random samples from a uniform distribution.

Samples are uniformly distributed over the half-open interval [low, high) (includes low, but excludes high).

Parameters low (float or NDAarray, optional) – Lower boundary of the output interval. All values generated will be greater than or equal to low. The default value is 0.

high (float or NDArray, optional) – Upper boundary of the output interval. All values generated will be less than high. The default value is 1.0.

shape (int or tuple of ints, optional) – The number of samples to draw. If shape is, e.g., (m, n) and low and high are scalars, output shape will be (m, n). If low and high are NDArrays with shape, e.g., (x, y), then output will have shape (x, y, m, n), where m*n samples are drawn for each [low, high) pair.

dtype ({'float16', 'float32', 'float64'}, optional) – Data type of output samples. Default is 'float32'

ctx (Context, optional) – Device context of output. Default is current context. Overridden by low.context when low is an NDArray.

out (NDArray, optional) – Store output to an existing NDArray.

Returns An NDArray of type dtype. If input shape has shape, e.g., (m, n) and low and high are scalars, output shape will be (m, n). If low and high are NDArrays with shape, e.g., (x, y), then the return NDArray will have shape (x, y, m, n), where m*n uniformly distributed samples are drawn for each [low, high) pair.

Return type NDArray

```
In [256... mx->random->seed(1);  
print nd->random->uniform(0, 1)->asstr;
```

```
[0.5238]
```

```
Out[256]: 1
```

```
In [257... mx->random->seed(1);  
print nd->random->uniform(0, 1, ctx=>mx->cpu(1));
```

```
<AI::MXNet::NDArray 1 @cpu(1)>
```

```
Out[257]: 1
```

```
In [258... mx->random->seed(1);  
print nd->random->uniform(-1, 1, shape=>[2,])->asstr;  
# [0.0476667 -0.889973]
```

```
[ 0.0477 -0.8900]
```

```
Out[258]: 1
```

```
In [259... mx->random->seed(1);  
my $low = nd->array([1,2,3]);  
my $high = nd->array([2,3,4]);  
print nd->random->uniform($low, $high, shape=>2)->asstr;  
# [[1.52383 1.05501]  
# [2.03996 2.59453]  
# [3.18597 3.69035]]
```

```
[[1.5238 1.0550]  
 [2.0400 2.5945]  
 [3.1860 3.6904]]
```

```
Out[259]: 1
```

nd->randint()

(low, high, shape=_Null, dtype=_Null, ctx=None, out=None, **kwargs)[source]

Draw random samples from a discrete uniform distribution.

Samples are uniformly distributed over the half-open interval [low, high) (includes low, but excludes high).

Parameters low (int, required) – Lower boundary of the output interval. All values generated will be greater than or equal to low.

high (int, required) – Upper boundary of the output interval. All values generated will be less than high.

shape (int or tuple of ints, optional) – The number of samples to draw. If shape is, e.g., (m, n) and low and high are scalars, output shape will be (m, n).

dtype ({'int32', 'int64'}, optional) – Data type of output samples. Default is 'int32'

ctx (Context, optional) – Device context of output. Default is current context. Overridden by low.context when low is an NDArray.

out (NDArray, optional) – Store output to an existing NDArray.

Returns An NDArray of type dtype. If input shape has shape, e.g., (m, n), the returned NDArray will shape will be (m, n). Contents of the returned NDArray will be samples from the interval [low, high).

Return type NDArray

<https://mxnet.apache.org/versions/1.6/api/r/docs/api/mx.nd.random.randint.html>

```
In [260... mx->random->seed(1);  
print nd->random->randint(5, 100)->asstr;  
# [61]
```

[61]

Out[260]: 1

```
In [261... mx->random->seed(1);  
print nd->random->randint(-10, 2, ctx=>mx->cpu(1))->asstr;  
# [ -2]
```

[-2]

Out[261]: 1

```
In [262... mx->random->seed(1);  
print nd->random->randint(-10, 10, shape=>[2,])->asstr;  
# [6 -2]
```

[6 -2]

Out[262]: 1

nd->random_choice

my \$sample = \$mx->random_choice(\$a, %args);

Generates a random sample from a given 1-D array or scalar population.

<https://numpy.org/doc/stable/reference/generated/numpy.random.choice.html>

A random sample is drawn from the elements of a 1-D array-like object or generated as an interval sequence if an integer population size is provided.

Sampling can be performed with or without replacement, and uniform or

custom probability distributions can be applied.

Similar function in the MXNet ndarray as `numpy.random.choice`.

See Also

<https://numpy.org/doc/stable/reference/generated/numpy.random.choice.html>.

Parameters

`$x` : Int | ARRAY ref | NDArray

The population from which to draw samples.

If an integer, the random sample is generated as if it were a sequence from 0 to $x - 1$. If an array reference or NDArray, samples are drawn directly from its elements.

`:$size=` : Int | ARRAY ref, optional

Output shape of the generated tensor.

If the given shape is, e.g., $[m, n, k]$, then $m * n * k$ samples are drawn. Default is undef, in which case a single scalar item (wrapped as a 0-D NDArray) is returned.

`:$replace=1` : Bool, optional

Whether the sample is drawn with or without replacement.

If true (default), values from the population can be selected multiple times.

`:$p=` : ARRAY ref | NDArray, optional

The probabilities associated with each entry in `$x`.

If not given, a uniform distribution over all entries in `$x` is assumed. If provided, the vector is automatically normalized internally to sum to 1.0.

`:$device=` : Context, optional

The execution context device where the random sampling kernel will be allocated and executed. Default is `AI::MXNet->cpu(0)`.

`:$out=` : NDArray, optional

An optional pre-allocated output tensor where the resulting samples will be written.

Returns

`$out` : NDArray

The generated random samples encapsulated in an NDArray container shaped according to the `:$size` parameter.

Raises

ValueError

If the required positional argument `$x` is missing or undefined.

If `$x` is an integer less than or equal to zero.

If `$x` or `:$p` are not 1-dimensional vectors.

If `$x` and `:$p` have different lengths.

If `:$replace` is false (0) and the requested total sample size is greater than the population size.

Notes

This implementation utilizes highly optimized internal MXNet bindings mapping directly to the native C-API operator::

`_npi_choice`

If no custom weights are provided, an empty input array is passed to the execution graph to trigger uniform distribution pipelines. Otherwise, the normalized probability tensor is injected dynamically.

```
In [263... # =====
# CASO 1: Comportamiento básico con Escalares
# =====

# 1.1 Si no se especifica 'size', debe retornar un escalar nativo de Perl
my $res = nd->random_choice(5);
is($res->size, 1, "Caso 1.1: Sin 'size' retorna un escalar");
ok($res->asscalar >= 0 && $res->asscalar < 5, "Caso 1.1: El escalar generado está en

ok 9 - Caso 1.1: Sin 'size' retorna un escalar
ok 10 - Caso 1.1: El escalar generado está en el rango [0, 4]

Out[263]: 1
```

```
In [264... # 1.2 Especificando un size entero (escalar)
my $array_res = nd->random_choice(5, size => 3);
isa_ok($array_res, 'AI::MXNet::NDArray', "Caso 1.2: Con 'size => 3' retorna un objeto
is($array_res->shape->[0], 3, "Caso 1.2: El tamaño del tensor unidimensional es exact

ok 11 - 'Caso 1.2: Con 'size => 3' retorna un objeto NDArray' isa 'AI::MXNet::NDArray'
ok 12 - Caso 1.2: El tamaño del tensor unidimensional es exactamente 3

Out[264]: 1
```

```
In [265... # 1.3 Especificando un size multidimensional (tupla/arrayref)
my $matrix_res = nd->random_choice(10, size => [2, 4]);
is_deeply($matrix_res->shape, [2, 4], "Caso 1.3: Con 'size => [2, 4]' respeta la estr

ok 13 - Caso 1.3: Con 'size => [2, 4]' respeta la estructura dimensional del tensor

Out[265]: 1
```

```
In [266... # =====
# CASO 2: Muestreo desde un Array-like (Tensores y Arreglos de Perl)
# =====

my @choices = (100, 200, 300, 400, 500);

# 2.1 Muestreo pasando un arrayref de Perl directamente
my $res = nd->random_choice(\@choices, size => 10);
printf "%s\n", $res->asstr;
# Verificar que todos los elementos seleccionados pertenezcan al conjunto original
my $all_valid = !grep { my $v = $_; !grep { $_ == $v } @choices } $res->tolist;
ok($all_valid, "Caso 2.1: Todos los elementos muestreados existen en el conjunto dado

[500 200 500 200 400 500 100 300 500 300]
ok 14 - Caso 2.1: Todos los elementos muestreados existen en el conjunto dado

Out[266]: 1
```

```
In [267... # =====
# CASO 3: Restricciones de Reemplazo (replace => 0)
# =====

# 3.1 Muestreo sin reemplazo: no deben existir duplicados si size == población
my $no_replace = nd->random_choice(5, size => 5, replace => 0);
# Usamos el truco de pasar a un hash para contar elementos únicos
my %unique_elements = map { $_ => 1 } $no_replace->tolist;
is(scalar(keys %unique_elements), 5, "Caso 3.1: 'replace => 0' no genera duplicados (

ok 15 - Caso 3.1: 'replace => 0' no genera duplicados (muestreo único completo)

Out[267]: 1
```

```
In [268... # =====
# CASO 4: Distribución de Probabilidades Ponderada (p => [...])
# =====

# 4.1 Probabilidad sesgada: el índice 2 tiene el 100% de probabilidad
my $biased_p = [0, 0, 1, 0, 0];
my $biased_res = nd->random_choice(5, size => 20, p => $biased_p);
my %biased_counts = map { $_ => 1 } @{$biased_res->asarray};
is_deeply([keys %biased_counts], [2],
    "Caso 4.1: Con p=[0,0,1,0,0], el 100% de las muestras fueron el índice 2");
```

ok 16 - Caso 4.1: Con p=[0,0,1,0,0], el 100% de las muestras fueron el índice 2

Out[268]: 1

```
In [269... # =====
# CASO 5: Validación Extrema de Errores (Esperando excepciones 'die')
# =====

# 5.1 Error cuando la muestra es mayor que la población sin reemplazo
eval {
    nd->random_choice(5, size => 10, replace => 0);
};
like($@, qr/Cannot take a larger sample than population/i,
    "Caso 5.1: Lanza correctamente un ValueError si size > a cuando replace es False")
```

ok 17 - Caso 5.1: Lanza correctamente un ValueError si size > a cuando replace es False

Out[269]: 1

```
In [270... # 5.2 Error cuando las dimensiones de 'p' no coinciden con 'a'
eval {
    nd->random_choice(5, size => 2, p => [0.5, 0.5]); # p tiene longitud 2, 'a' es 5
};
like($@, qr/a and p must have same size/i,
    "Caso 5.2: Lanza correctamente un ValueError si las longitudes de 'a' y 'p' difieren")
```

ok 18 - Caso 5.2: Lanza correctamente un ValueError si las longitudes de 'a' y 'p' difieren

Out[270]: 1

```
In [271... # 5.3 Error cuando 'a' es un escalar negativo o cero
eval {
    nd->random_choice(0, size => 5);
};
like($@, qr/a must be greater than 0/i,
    "Caso 5.3: Lanza correctamente un ValueError si 'a' <= 0");
```

ok 19 - Caso 5.3: Lanza correctamente un ValueError si 'a' <= 0

Out[271]: 1

```
In [272... # =====
# CASO 6: Caso Límite de Tamaño Cero (Size => 0)
# =====

# 6.1 NumPy devuelve un arreglo vacío si el tamaño acumulado de size da 0
my $zero_size_res = nd->random_choice(5, size => [3, 0, 2]);
is_deeply($zero_size_res->shape, [3, 0, 2],
    "Caso 6.1: Soporta dimensiones de tamaño cero regresando la estructura vacía");
```

ok 20 - Caso 6.1: Soporta dimensiones de tamaño cero regresando la estructura vacía

Out[272]: 1

```
In [273... # Generate a uniform random sample from nd->arange(5) of size 3:
nd->print(nd->random_choice(5, 3));
# This is equivalent to nd->random->randint(0,5,3)
```

```
[2 1 2]
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray int64
```

Out[273]: 1

```
In [274... # Generate a non-uniform random sample from np.arange(5) of size 3:
nd->print(nd->random_choice(5, 3, p=>[0.1, 0, 0.3, 0.6, 0]));
# [2 3 3] # random
```

```
[2 0 3]
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray int64
```

Out[274]: 1

```
In [275... # Generate a uniform random sample from np.arange(5) of size 3 without replacement:
nd->print(nd->random_choice(5, 3, replace=>0));
# array([3,1,0]) # random
# This is equivalent to nd->arange(5)->shuffle->slice([0,3]);
```

```
[0 1 3]
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray int64
```

Out[275]: 1

```
In [276... # Generate a non-uniform random sample from np.arange(5) of size 3 without replacemen
nd->print(nd->random_choice(5, 3, replace=>0, p=>[0.1, 0, 0.3, 0.6, 0]));
# [2 3 0] # random
```

```
[3 2 0]
<AI::MXNet::NDArray 3 @cpu(0)> AI::MXNet::NDArray int64
```

Out[276]: 1

2.1.6. Conversion to Other Perl Objects

Converting to a tensor (NDarray), or vice versa, is easy. The converted result does not share memory. This minor inconvenience is actually quite important: when you perform operations on the CPU or on GPUs, you do not want to halt computation, waiting to see whether the Perl array might want to be doing something else with the same chunk of memory.

```
In [277... my $A = [1, 2, 3]; # Perl array reference
my @B = (4, 5, 6); # Perl array list
print ref ($A);
```

ARRAY

Out[277]: 1

```
In [278... my $C = nd->array($A); # Obtain a NDArray out of a Perl array reference
print ref($C), "\n";
print $C->asstr;
```

```
AI::MXNet::NDArray
[1 2 3]
```

Out[278]: 1

```
In [279... my $D = nd->array(\@B, dtype => 'int32'); # Obtain a NDArray out of a Perl array list
print ref($D), "\n";
print $D->asstr;
print "\n", $D->dtype;
```

```
AI::MXNet::NDArray  
[4 5 6]  
int32
```

Out[279]: 1

```
In [280... my $E = $C->asarray; # Obtain a Perl array reference out of a NDArray  
print ref($E), "\n";  
print dump $E;
```

```
ARRAY  
[1, 2, 3]
```

Out[280]: 1

```
In [281... my @F = $C->tolist; # Obtain a Perl list out of NDArray  
print dump @F;
```

```
(1, 2, 3)
```

Out[281]: 1

Other types of conversions

To convert a size-1 tensor to a Perl scalar, we can invoke the `->asscalar` attribute.

```
In [282... my $a = nd->array([3.5]);
```

Out[282]: <AI::MXNet::NDArray 1 @cpu(0)>

```
In [283... print $a, "\n", $a->dtype, "\n", $a->asscalar;
```

```
<AI::MXNet::NDArray 1 @cpu(0)>  
float32  
3.5
```

Out[283]: 1

We can see 3 mini-batches of data (and labels), the first ones with 3 samples, whereas the last one with 1 sample, which makes sense given we started with a dataset of 10 samples. When comparing the shape of the batches to the samples returned by the Dataset, we've gained an extra dimension at the start which is sometimes called the batch axis.

Our `data_loader` loop will stop when every sample of dataset has been returned as part of a batch. Sometimes the dataset length isn't divisible by the mini-batch size, leaving a final batch with a smaller number of samples.

`DataLoader`'s default behavior is `keep` to return this smaller mini-batch, but this can be changed by setting the `last_batch` parameter to `discard` (which ignores the last batch) or `rollover` (which starts the next epoch with the remaining samples).

2.1.10. Complementary functions for NDArray manipulation

```
In [284... use AI::MXNet::Base; # Available functions product enumerate assert zip check_call bu  
# https://metacpan.org/pod/AI::MXNet::Base
```

```
In [285... my $x = nd->arange(5);  
my $y = $x + 5;  
print $x->asstr, " ", $y->asstr, "\n";
```

```
[0 1 2 3 4] [5 6 7 8 9]
```

Out[285]: 1

```
In [286... for (zip($x, $y)) {  
    my ($x1, $y1) = @$_;  
    print $x1->asstr, " ", $y1->asstr, "\n";  
}
```

```
[0] [5]  
[1] [6]  
[2] [7]  
[3] [8]  
[4] [9]
```

```
In [287... printf "%s %s\n", $_->[0]->asstr, $_->[1]->asstr for (zip($x, $y));
```

```
[0] [5]  
[1] [6]  
[2] [7]  
[3] [8]  
[4] [9]
```

```
In [288... print map {$_->[0]->asstr, " ", $_->[1]->asstr, "\n";} zip($x, $y)
```

```
[0] [5]  
[1] [6]  
[2] [7]  
[3] [8]  
[4] [9]
```

Out[288]: 1

```
In [289... for (enumerate($x)){  
    my ($i, $x1) = @$_;  
    printf "%d %s\n", $i, $x1->asstr;  
}
```

```
0 [0]  
1 [1]  
2 [2]  
3 [3]  
4 [4]
```

```
In [290... printf "%d %s\n", $_->[0], $_->[1]->asstr for (enumerate($x));
```

```
0 [0]  
1 [1]  
2 [2]  
3 [3]  
4 [4]
```

```
In [291... print map { sprintf("%d %s\n", $_->[0], $_->[1]->asstr) } enumerate($x);
```

```
0 [0]  
1 [1]  
2 [2]  
3 [3]  
4 [4]
```

Out[291]: 1

2.1.7. Gluon Datasets and DataLoader

Dataset objects are used to represent collections of data, and include methods to load and parse the data (that is often stored on disk). Gluon has a number of different Dataset classes for working with image data straight out-of-the-box, but we'll use the `ArrayDataset` to introduce the idea of a Dataset.

We first start by generating random data X (with 2 variables) and corresponding random labels y to simulate a typical supervised learning task. We generate 10 samples and we pass them all to the ArrayDataset.

```
In [292... my $X = nd->arange(20)->reshape([10, 2]);
mx->random->seed(1);
my $y = nd->random->randint(0, 2, shape=>[10, 1]);
my $dataset = mx->gluon->data->ArrayDataset(data => $X, label => $y);

printf "Original X for training: %s\n", $X->asstr;
printf "Original y training: %s\n", $y->asstr;

my $train_data = mx->gluon->data->DataLoader($dataset, batch_size => 3, shuffle => 1,

# The number of batches can be obtained by its property len().
print "num_batches: ", $train_data->len();
```

Original X for training: [[0 1]

[2 3]
[4 5]
[6 7]
[8 9]
[10 11]
[12 13]
[14 15]
[16 17]
[18 19]]

Original y training: [[0]

[0]
[1]
[1]
[0]
[1]
[1]
[1]
[0]
[1]]

num_batches: 3

Out[292]: 1

```
In [293... # Minibatches of the training dataset
for (my ($i, $batch, $X, $y) = 0; eval{ $batch = <$train_data>, ($X, $y) = @$batch};
    print "X:", $X->asstr;
    print "y:", $y->asstr, "\n";
}
```

```
In [294... my $X = nd->arange(start=>20, stop=>30)->reshape([5, 2]);
mx->random->seed(2);
my $y = nd->random->randint(0, 2, shape=>[5, 1]);
my $dataset = mx->gluon->data->ArrayDataset(data => $X, label => $y);

printf "Original X for testing: %s\n", $X->asstr;
printf "Original y testing: %s\n", $y->asstr;

my $test_data = mx->gluon->data->DataLoader($dataset, batch_size => 3, shuffle => 0,

# The number of batches can be obtained by its property len().
printf "num_batches: %d\n\n", $test_data->len();
```

```
Original X for testing: [[20 21]
 [22 23]
 [24 25]
 [26 27]
 [28 29]]
Original y testing: [[1]
 [0]
 [1]
 [1]
 [1]]
num_batches: 2
```

```
Out[294]: 1
```

```
In [295... # Minibatches of the test dataset

for (my ($i, $batch, $X, $y) = 0; eval{ $batch = <$test_data>, ($X, $y) = @$batch}; $
    printf "X[%d]: %s\n", $i, $X->asstr;
    printf "y[%d]: %s\n", $i, $y->asstr;
}
```

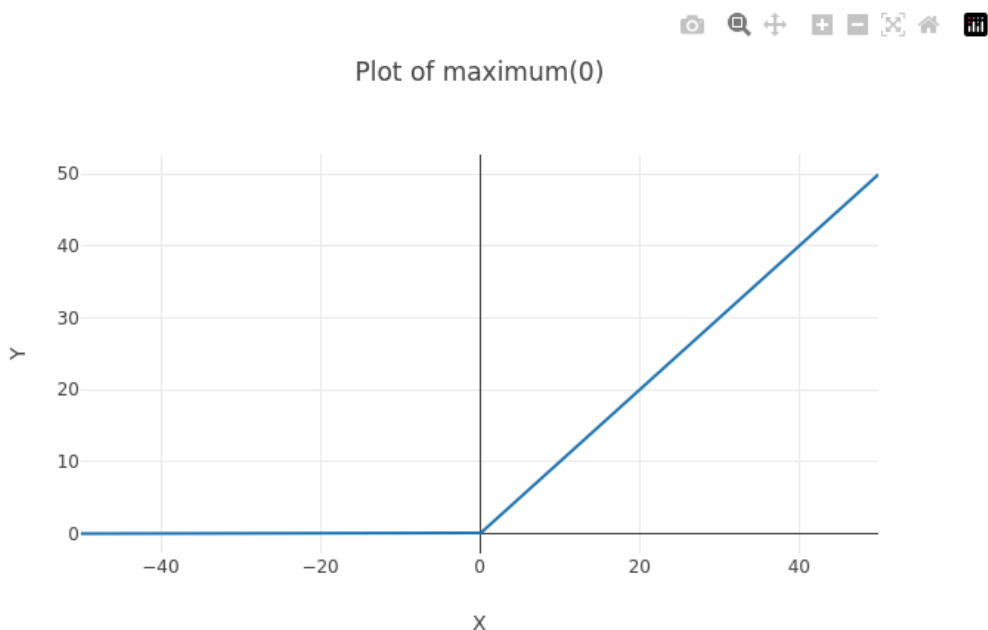
Plots/Graphics

```
In [296... # nd->maximum_scalar($x, 0)

$x = nd->arange(start=>-50, stop=>50, step=>0.1);

my $trace = [new Chart::Plotly::Trace::Scatter(x => $x->asarray,
                                                y => nd->maximum_scalar($x, 0)->asarra
                                                name => 'model',
                                                mode => 'lines')];

my $plot = new Chart::Plotly::Plot(traces => $trace,
                                   layout => {title => {text => 'Plot of maximum(0)'},
                                             xaxis => {title => 'X'}, yaxis => {title => 'Y'}})
sml->embedplot($plot, width=>900, height=>450);#show_plot($plot);IPerl->display($plot
```

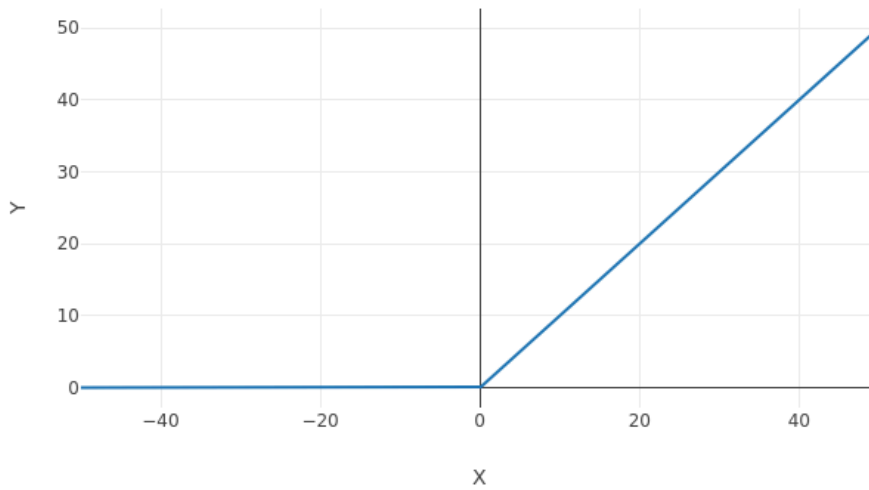
[illegible]

```
mode => 'lines']];
```

```
my $plot = new Chart::Plotly::Plot(traces => $trace,  
    layout => {title => {text => 'Plot of relu()'},  
                xaxis => {title => 'X'}, yaxis => {title => 'Y'}})  
sml->embedplot($plot, width=>900, height=>450);
```



Plot of relu()

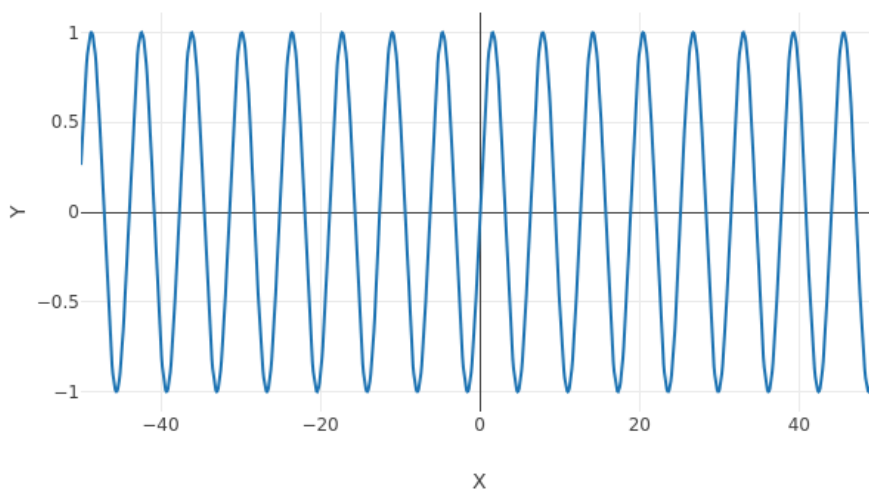


In [298...

```
my $trace = [new Chart::Plotly::Trace::Scatter(x => $x->asarray,  
    y => $x->sin()->asarray,  
    name => 'model',  
    mode => 'lines')];  
  
$plot = new Chart::Plotly::Plot(traces => $trace,  
    layout => {title => {text => 'Plot of sin(x)'},  
                xaxis => {title => 'X'}, yaxis => {title => 'Y'}})  
sml->embedplot($plot, width=>900, height=>450);
```



Plot of sin(x)



In [299...

```
print nd->reverse($x, axis=>0)->slice([undef, 10])->asstr('%.2f');
```

```
[49.90 49.80 49.70 49.60 49.50 49.40 49.30 49.20 49.10    49]
```

Out[299]: 1

In [300...

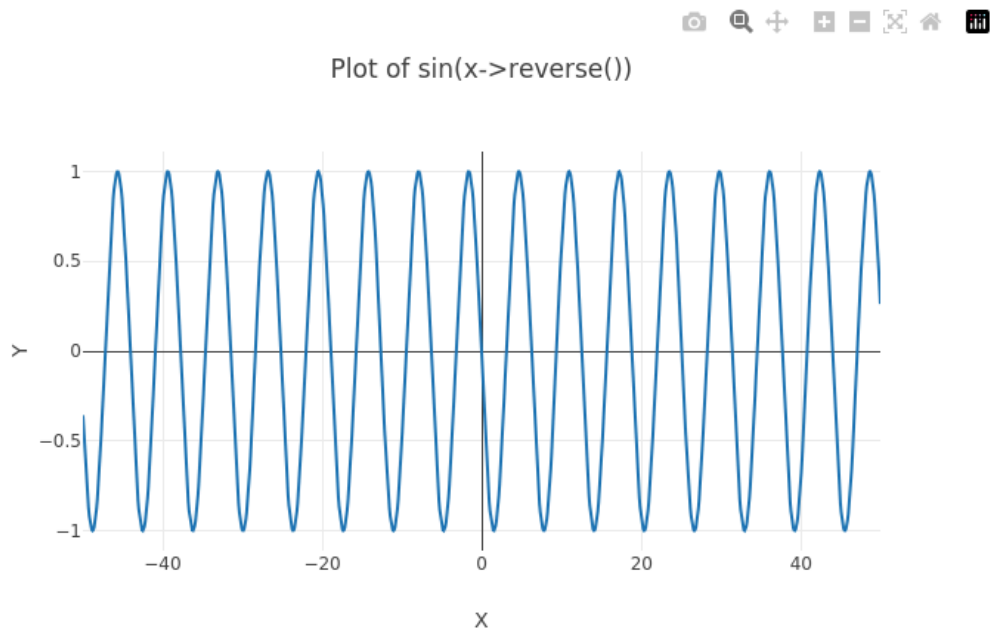
```
my $trace = [new Chart::Plotly::Trace::Scatter(x => $x->asarray,  
    y => nd->reverse($x->sin(), axis=>0)->
```

```

name => 'model',
mode => 'lines']];

$plot = new Chart::Plotly::Plot(traces => $trace,
                                layout => {title => {text => 'Plot of sin(x->reverse())'},
                                             xaxis => {title => 'X'}, yaxis => {title => 'Y'}})
sml->embedplot($plot, width=>900, height=>450);

```



2.1.9. Summary

- The main interface to store and manipulate data for deep learning is the tensor (n-dimensional array). It provides a variety of functionalities including basic mathematics operations, broadcasting, indexing, slicing, memory saving, and conversion to other Perl objects.

```

In [301]... # Finalizar reporte de pruebas para Jupyter
Test::More::done_testing();

```

1..20

Out[301]: 1

In []: